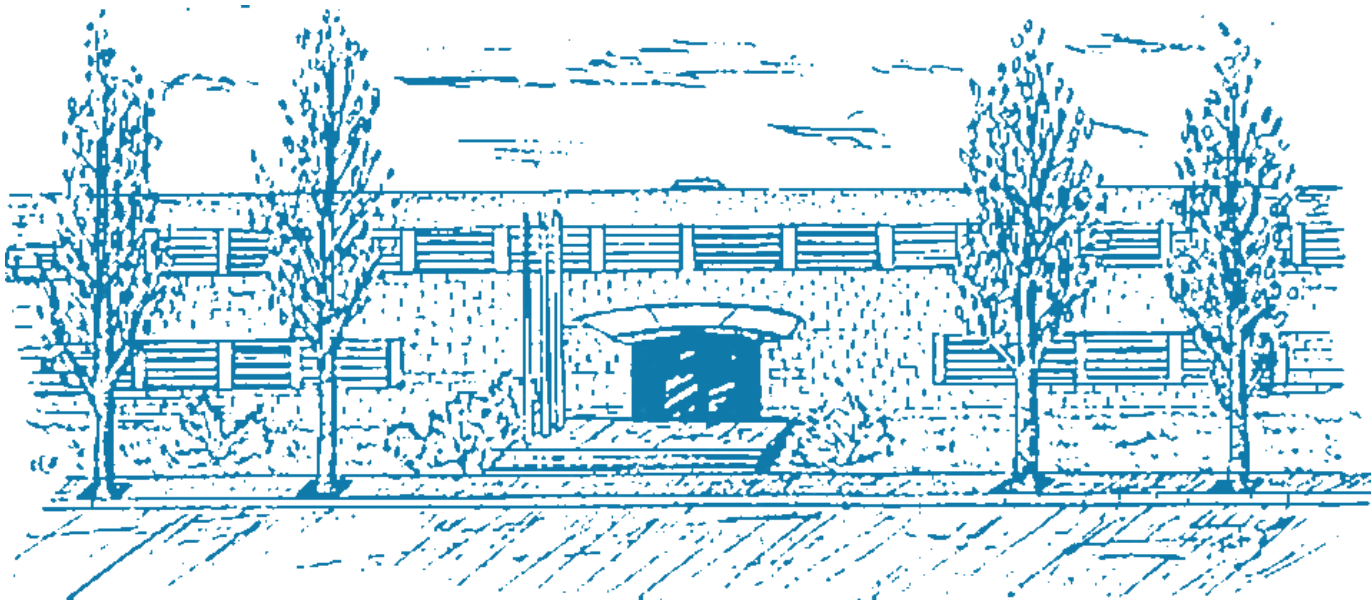


UNIVERSITAT POLITÈCNICA DE CATALUNYA - BarcelonaTech (UPC)
FME - FACULTAT DE MATEMÀTIQUES I ESTADÍSTICA

MASTER IN ADVANCED MATHEMATICS AND MATHEMATICAL ENGINEERING (MAMME)
MASTER'S THESIS

Stability Analysis and Neural Network Modelling of Clustered Large-Scale Brain Dynamics



Author: Ferran Estrella Serra

Director: Manuel Molano Mazón

Co-director: Gemma Huguet Casades

Department: Department of Mathematics

Barcelona, 28/05/2026



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat de Matemàtiques i Estadística



Universitat Politècnica de Catalunya
Facultat de Matemàtiques i Estadística

Master in Advanced Mathematics and Mathematical Engineering
Master's thesis

Stability Analysis and Neural Network Modelling of Clustered Large-Scale Brain Dynamics

Ferran Estrella Serra

Supervised by Manuel Molano Mazón and Gemma Huguet Casades

May 2026

Abstract

In this thesis, we study large-scale brain dynamics, their reduction through clustering methods, their stability properties, and their approximation using neural operator learning techniques. We first introduce a large-scale brain model produced by coupling next-generation neural mass models, using a network topology derived from structural connectomics data. This leads to a parameter-dependent dynamical system governing neuronal activity. Then, we consider a dimensionality reduction of the empirical structural connectivity matrix of 90 nodes using spectral clustering. The resulting clustered connectivity matrix, of four nodes, provides a reduced-dimensional representation of the network, used to analyze whether the clustered system preserves stability properties of the original network. Later, we perform a linear stability analysis of homogeneous solutions in the clustered large-scale brain model, depending on the external input parameter I_{ext}^E and the coupling strength ε . Specifically, we relate the stability properties of homogeneous equilibrium points and periodic orbits to the spectrum of the clustered structural connectivity through dispersion relations and master stability functions, respectively. This framework is applied to obtain bifurcation diagrams and stability regions in parameter space. Results are compared with those obtained for the original connectivity matrix, observing that relevant stability properties are preserved over a range of parameters, except for a specific region. Finally, we introduce the DeepONet neural network architecture for approximating solutions of parameter-dependent initial value problems. We apply it to a phase-amplitude coupling system describing local dynamics near a stable homogeneous periodic orbit. Numerical experiments compare DeepONet predictions with classical numerical solutions and demonstrate its performance in both forward prediction and inverse problem settings.

Keywords

Quantitative Biology, Computational Neuroscience, Dynamical Systems, Neural Mass Models, Phase-Amplitude Coupling, Differential Equations, Numerical Methods, Deep Learning, Neural Operator Learning, DeepONet

Contents

1	Introduction	6
2	Large-Scale Brain Model	8
2.1	Connectivity and Node Scales	8
2.1.1	Connectivity Scale	8
2.1.2	Node Scale	9
2.2	Large-Scale Brain Dynamics	10
3	Structural Connectivity Clustering	12
3.1	Spectral Clustering Method	12
3.1.1	Definitions	12
3.1.2	Algorithm	13
3.1.3	Clustering Quality	13
3.2	Clustering of the Empirical Structural Connectivity	13
4	Linear Stability Analysis of Homogeneous Solutions	16
4.1	Homogeneous Invariant Manifold	16
4.2	Linear Stability Analysis of Homogeneous Solutions	17
4.2.1	Structural Connectivity Spectrum	17
4.2.2	Perturbation Projection	17
4.2.3	Dispersion Relation	18
4.2.4	Master Stability Function	19
4.3	Computation of the Dispersion Relations and Master Stability Functions for the Clustered System	20
4.4	Bifurcation Diagram Computation	22
4.5	Comparison with the Original Structural Connectivity	24
5	DeepONet Predictions for Phase-Amplitude Dynamics	27
5.1	Motivation of the Problem	27
5.2	Phase-Amplitude Coupling Dynamics	27
5.3	DeepONet Estimation of Solutions to the Parametrized Initial Value Problem	28
5.3.1	DeepONet Architecture Applied to Parametrized Functions	29
5.3.2	DeepONet Architecture Design Choices	30
5.4	DeepONet Training	32
5.4.1	Loss Optimization	32
5.4.2	Types of Loss Functions	33
5.5	DeepONet Prediction and the Inverse Problem	35
5.6	DeepONet Experimentation	35
5.6.1	Considered Phase-Amplitude Coupling Dynamics Initial Value Problem	35
5.6.2	DeepONet Architecture Refinements for Phase-Amplitude Coupling Dynamics	36
5.6.3	Hyperparameter Selection and Software Implementation	37
5.6.4	Experiment Results	38
A	K-Means Clustering Method	43

B	Clustering Metrics	45
B.1	Eigengap Heuristic	45
B.2	Q-Modularity	45
B.3	Silhouette Width	45
B.4	Calinski-Harabasz Index	46
C	Training Techniques	47
C.1	Adam Optimization Algorithm	47
C.2	Gradient Clipping	47
C.3	Xavier/Glorot Initialization	48

1. Introduction

A major goal in computational neuroscience is to understand how oscillatory and rhythmic activity emerges in brain dynamics. Current research seeks to address this problem by proposing mathematical models that describe and simulate complex large-scale brain dynamics. Recent advances in the field use next-generation neural mass models (NG-NMMs) to model brain activity, which are parameter-dependent dynamical systems providing accurate descriptions of the macroscopic activity of spiking neuronal networks [2, 3]. By coupling several of these NG-NMMs and incorporating network connectivities obtained from structural connectomics data, one constructs large-scale brain models in which different brain regions are represented by NG-NMMs, while the dynamical interactions between them are determined by the underlying network structure [1].

Nevertheless, these models involve major mathematical challenges due to high dimensionality and the presence of features including noise and heterogeneity. Such conditions commonly lead to limited interpretability of the system dynamics and significant computational difficulties.

Motivated by the goal of overcoming these complications, this project proposes a simplification of the large-scale brain model introduced in [1]. Precisely, it reduces the dimensionality of the associated structural connectivity network through a clustering methodology, and analyzes the effects of this dimensionality reduction on the system dynamics. Specifically, we explore the graph spectral clustering methodology [4] and apply it to the empirical structural connectivity introduced in [7]. We investigate how this clusterization affects the linear stability analysis of numerically obtained homogeneous equilibrium points and periodic orbits of the system, and compare the resulting stability properties with those reported in [1].

A second main focus of this project is to further investigate the homogeneous local system dynamics, specifically around stable homogeneous periodic orbits. Dimensionality reduction can be achieved in this setting by considering the phase-amplitude coupling dynamics of the brain model [9], in which the trajectories are described by a two-dimensional system governing the phase and amplitude relative to the periodic orbit.

However, in this second study, a different methodology is adopted. Motivated by the interest in investigating neural network capabilities to approximate neuronal activity in particular, and parameter-dependent dynamical systems in general, this work uses Deep Operator Networks (DeepONets) [10] as a neural network architecture capable of approximating solutions of such dynamical systems. In particular, we implement the DeepONet architecture to predict the dynamics of a phase-amplitude system, and assess its accuracy by comparing the approximated dynamics with those obtained using classical numerical methods.

We also explore additional predictive capabilities of DeepONet through the inverse problem setting, where we use the trained DeepONet to infer the underlying dynamical parameters from noisy trajectory datasets.

The project is structured into five main chapters. Each chapter is divided into several sections, typically beginning with a theoretical exposition followed by the application of the developed theory to the specific problem considered in that chapter. We remark that the project not only focuses on the obtained results, but also on the underlying mathematical modelling that supports them. In particular, it aims to serve as a self-contained reference with a clear, rigorous, and didactic mathematical exposition. Mathematical theory which is not explicitly developed or referenced is assumed to be known by the reader, and this mainly includes linear algebra, differential equations and dynamical systems theory, numerical methods, and some introductory background in machine learning and deep learning.

We now provide a brief description of each section and the main results obtained.

Chapter 2 introduces the large-scale brain model, its interpretable structure, and the parameter-dependent dynamical system. Chapter 3 presents the spectral clustering method and applies it to the empirical structural connectivity network to obtain a clustered connectivity with 4 nodes.

Next, Chapter 4 performs a linear stability analysis of homogeneous periodic orbits and equilibrium points of the clustered system, depending on two relevant parameters of the system dynamics. In particular, it explores the relation between the stability characteristic exponents of each homogeneous solution and the spectrum of the clustered connectivity, through dispersion relations and master stability functions. It analyzes the stability properties of the clustered model via a bifurcation diagram and describes the stability characteristic exponents as a function of the considered parameter pairs. It concludes by comparing the stability results for the clustered connectivity with those obtained for the original connectivity [1], showing that relevant stability properties of the homogeneous solutions are preserved over an explored range of parameters, except in a specific parameter region.

The last Chapter 5 presents the phase-amplitude coupling system for modelling dynamics around stable homogeneous periodic orbits. It introduces the DeepONet methodology, including its architecture, training, prediction, and adaptation to initial value problem solution approximation (specifically, to the phase-amplitude system). Finally, it presents results on the performance of DeepONet in predicting solutions of the phase-amplitude coupling system. In particular, it evaluates the accuracy of the DeepONet predictions compared with numerically integrated solutions, and highlights its capability to fit noisy trajectory data to the dynamical model considered during training.

Additional mathematical details and technical derivations are provided in the appendices.

Several directions for future work can be considered. These include, for instance, an exploration of the clustered system dynamics, in particular the classification of stationary states. As an intersection of the two main lines of work developed in this project, it could also be of interest to study phase–amplitude coupling using DeepONet predictions in the full network system. More ambitiously, one could extend this approach to study the full clustered network dynamics using DeepONets.

The implementation of the numerical experiments, DeepONet training, and the associated results carried out in this work have been developed in Python. The full codebase used to generate the results presented in this thesis is publicly available in the following GitHub repository: <https://github.com/FerranEstrella/TFM>.

We remark that the code corresponding to Chapters 2 and 4 has been adapted from the code available in [1]. The code for Chapters 3 and 5 has been developed by the author.

2. Large-Scale Brain Model

In this chapter, we present the mathematical model describing large-scale neuronal activity in the brain. We begin by introducing the structure of the model at two different levels: a global and a local scale, which rely on two fundamental components of the large-scale brain framework, namely the connectivity matrix and neural mass modelling, respectively. We conclude the chapter by presenting the dynamical system governing the neuronal activity, along with the relevant parameters that define its behaviour.

2.1 Connectivity and Node Scales

In this section, we introduce the two interpretability scales of the considered large-scale brain model, namely:

- The global scale, referred to as the *connectivity scale*.
- The local scale, referred to as the *node scale*.

We describe both scales in detail to provide insight into the construction and understanding of the dynamical system governing neuronal activity, which is presented in the next section.

2.1.1 Connectivity Scale

The connectivity scale of the model is the global level at which different volumetric brain regions are represented as nodes, and the interconnections between them as edges. Together, they constitute an undirected graph, which we refer to as *structural connectivity graph*. We denote by N the number of nodes in the structural connectivity graph, also referred to as the number of populations, since each node is treated as a distinct population in the dynamical system describing neuronal activity. We consider an $N \times N$ connectivity matrix w representing the structural connectivity graph, where each entry w_{ij} encodes the strength of the connection between nodes i and j , also referred to as long-range interactions. The matrix w is symmetric, has non-negative entries, and a null diagonal.

The connectivity matrix w is derived from empirical data on the anatomical connectivity of the brain. Specifically, diffusion MRI tractography is used to estimate white-matter fibre pathways, which are then mapped into a connectivity matrix via a parcellation of the brain into regions of interest (Figure 1). Each node corresponds to one region of interest, and the entry w_{ij} typically reflects the strength of the reconstructed fibre connectivity between regions i and j (for instance, proportional to the number of streamlines, possibly normalized by region size or tract length, and averaged across different measurements). For example, a widely used parcellation is the AAL90 atlas, which yields a structural connectivity matrix with $N = 90$ nodes. Other parcellation schemes provide finer spatial resolution, using a larger number of regions—typically on the order of a few hundred (e.g. 200–400 nodes).

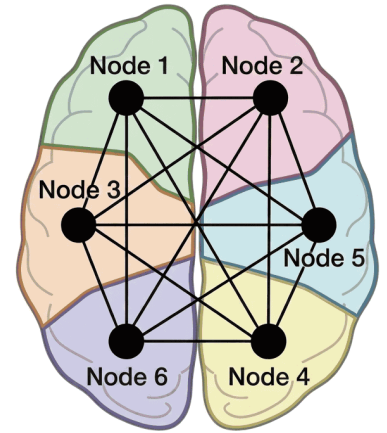


Figure 1: Schematic illustration of the structural connectivity graph linking different brain regions, each represented by a distinct node.

In this project, we consider an empirical structural connectivity graph obtained using the AAL90 parcellation over tractography data from dMRI, averaged across 16 human subjects. This tractography dataset was obtained in [7], and is the same as that used in [1].

Specifically, we will cluster the connectivity matrix associated with this dataset and use the resulting clustered graph as the connectivity structure describing the interactions between nodes.

2.1.2 Node Scale

At the node scale, we describe the local dynamics of each neuronal population within each brain region, that is, at each node.

The local dynamics at each node are described by a *next-generation neural mass model* (NG-NMM). This model captures the evolution of the mean firing rate and membrane potential of a population of quadratic integrate-and-fire (QIF) neurons, composed of excitatory (E) and inhibitory (I) subpopulations that interact through exponentially decaying synapses.

The model relies on the following simplifications:

- The system is deterministic; no noise is included in the local neuronal activity.
- The system is homogeneous, meaning that neuronal activity is assumed to be uniform across each brain region (while still distinguishing between excitatory (E) and inhibitory (I) subpopulations).
- No transmission delays are included to model the intra-population connections between neurons within each subpopulation.
- Neurons within each population interact with all other neurons in the same population (all-to-all connectivity).

Neural mass models constitute a primary approach to modelling neuronal activity, providing low-dimensional descriptions of population-level dynamics. They have been particularly successful in reproducing and fitting neuroimaging data (such as EEG and MEG), thereby establishing a principled link between microscopic neural mechanisms and macroscopic brain signals.

This model is referred to as *next generation* because it provides an exact and interpretable description of the macroscopic activity of spiking neural networks, unlike earlier models. By coupling multiple NG-NMMs through the structural connectivity, one obtains large-scale brain models capable of reproducing a broad repertoire of neuronal behaviours and complex emerging spatiotemporal patterns.

For instance, these models can generate fast oscillations whose amplitude is modulated by a slower rhythm, a form of cross-frequency coupling (CFC) that does not arise in isolated nodes.

2.2 Large-Scale Brain Dynamics

In this section, we describe the dynamical system modelling large-scale brain dynamics, together with the parameters considered in the model.

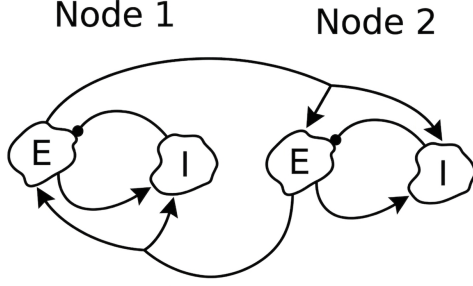


Figure 2: Schematic illustration of the mutual interactions between excitatory (E) and inhibitory (I) subpopulations within nodes, and between nodes via structural connectivity. Note that inhibitory populations do not interact across nodes.

Within each node, the excitatory and inhibitory subpopulations interact reciprocally: the excitatory population drives the inhibitory one, while the inhibitory population suppresses the excitatory one. Additionally, each excitatory subpopulation in a node interacts with both the excitatory and inhibitory populations of other nodes through the structural connectivity.

The dynamics of each node can be described by 6 variables:

- The mean firing rate of the excitatory and inhibitory subpopulations, r_E and r_I .
- The mean membrane potential of the excitatory and inhibitory subpopulations, v_E and v_I .
- The exponentially decaying synaptic activity of the excitatory and inhibitory subpopulations, s_E and s_I .

Since there are N nodes, the total number of variables is $6N$. Their evolution is described by the following system of autonomous differential equations:

$$\begin{aligned}
 (E_j) \quad & \begin{cases} \tau_E \dot{r}_{E,j} = \frac{\Delta_E}{\pi \tau_E} + 2r_{E,j}v_{E,j}, \\ \tau_E \dot{v}_{E,j} = \bar{\eta}_E + v_{E,j}^2 - (\tau_E \pi r_{E,j})^2 + I_{\text{ext}}^E + \tau_E (J_{EE}s_{E,j} - J_{EI}s_{I,j} + I_j), \\ \tau_{s_E} \dot{s}_{E,j} = -s_{E,j} + r_{E,j} \end{cases} \\
 (I_j) \quad & \begin{cases} \tau_I \dot{r}_{I,j} = \frac{\Delta_I}{\pi \tau_I} + 2r_{I,j}v_{I,j}, \\ \tau_I \dot{v}_{I,j} = \bar{\eta}_I + v_{I,j}^2 - (\tau_I \pi r_{I,j})^2 + I_{\text{ext}}^I + \tau_I (J_{IE}s_{E,j} - J_{II}s_{I,j} + I_j), \\ \tau_{s_I} \dot{s}_{I,j} = -s_{I,j} + r_{I,j} \end{cases} \quad (1)
 \end{aligned}$$

where:

- The variables $r_{\ell,j}$, $v_{\ell,j}$, and $s_{\ell,j}$ denote the mean firing rate, mean membrane potential, and synaptic current, respectively, of subpopulation $\ell \in \{E, I\}$ in node j .
- The parameters τ_ℓ and τ_{s_ℓ} represent the membrane and synaptic time constants of subpopulation $\ell \in \{E, I\}$.
- The parameters $J_{\ell k}$ denote the synaptic strength between neurons in subpopulations $\ell, k \in \{E, I\}$.
- The parameters $\bar{\eta}_\ell$ and Δ_ℓ , for $\ell \in \{E, I\}$, are the center and half-width of the Cauchy distribution, respectively, describing the input current at the single-neuron level.

- The parameters I_{ext}^ℓ denote external currents impinging on subpopulation $\ell \in \{E, I\}$.
- The variable I_j represents the long-range excitatory input received by node j from other populations through the structural connectivity, given by

$$I_j = \varepsilon \sum_{k=1}^N \tilde{w}_{jk} s_{E,k}, \quad \text{such that} \quad \sum_{k=1}^N \tilde{w}_{jk} = 1, \quad (2)$$

where ε is a global coupling parameter between nodes, and $\tilde{w}$ is the row-normalized connectivity matrix.

The normalized connectivity structure ensures that all units in the network receive the same total input, although distributed differently across nodes. This simplifies the analysis of the homogeneous state of the system, i.e., the state in which all nodes exhibit identical dynamics described by the variables (r_I, v_I, s_I) for $I \in \{E, I\}$. In this state,

$$I_j = \varepsilon \sum_{k=1}^N \tilde{w}_{jk} s_{E,k} = \varepsilon s_E \sum_{k=1}^N \tilde{w}_{jk} = \varepsilon s_E, \quad (3)$$

which is independent of the node index j . Therefore, the homogeneous state does not depend on the structural connectivity.

In this project, we primarily study the system under the influence of the external input to the excitatory population, I_{ext}^E , and the coupling strength ε . The remaining parameters are held fixed and are chosen according to [1]. These parameters are collected in Table 1.

Parameter	Value	Parameter	Value	Parameter	Value
τ_E	8	τ_I	8	τ_{sE}	1
τ_{sI}	5	$\bar{\eta}_E$	-5	$\bar{\eta}_I$	-5
Δ_E	1	Δ_I	1	J_{EE}	5
J_{EI}	13	J_{IE}	13	J_{II}	5
I_{ext}^E	not fixed	I_{ext}^I	0	ε	not fixed

Table 1: Parameter configuration used throughout this work. Time constants values are expressed in milliseconds.

3. Structural Connectivity Clustering

In this chapter, we present the spectral clustering methodology and its application to reduce the empirical structural connectivity matrix of 90 nodes to a clustered connectivity matrix with fewer nodes.

We are interested in a reduced-dimensional representation of the network because we aim to understand the role of the connectivity in the system dynamics. In particular, we want to study whether a system defined on a clustered network exhibits stability properties similar to those of the original network and, if so, which similarities and differences are the most relevant.

In addition, analyzing a low-dimensional network can yield more interpretable dynamics than those of the full network and can significantly accelerate numerical computations, since the dimensionality of the system is substantially reduced.

We begin the chapter with a section devoted to the spectral clustering methodology, including the reasons for selecting this clustering algorithm, its implementation for a general connectivity matrix, and the metrics used to assess clustering quality. The following section applies spectral clustering to the empirical connectivity matrix and studies which number of clusters yields the best clustering quality.

3.1 Spectral Clustering Method

In this section, we introduce the spectral clustering method.

Several approaches exist for clustering a connectivity matrix, or more generally, an undirected graph. In this work, we adopt *spectral clustering* due to its simplicity and interpretability.

In particular, spectral clustering appears to implicitly preserve the dominant eigenvalues of the connectivity matrix (as discussed later). Since these eigenvalues play a fundamental role in the stability properties of the system (as will be shown in the following chapter), a key goal of this study is to investigate whether spectral clustering is able to preserve these relevant stability characteristics. This consideration is also one of the main motivations for selecting this clustering method.

Further details on the underlying principles and theoretical foundations of this method can be found in [4].

3.1.1 Definitions

Let W be an $N \times N$ symmetric, non-negative real matrix with a null diagonal, representing the weight matrix of an undirected graph. We denote by D its associated $N \times N$ *degree matrix*, i.e., the diagonal matrix with entries $d_1, \dots, d_N$ given by

$$d_i = \sum_{j=1}^N W_{ij}. \quad (4)$$

We denote by

$$L_{rw} = D^{-1}L = I - D^{-1}W \quad (5)$$

the *normalized random-walk Laplacian* of W , where $L = D - W$ is the *unnormalized Laplacian* and I is the identity matrix. We consider the random-walk Laplacian because it has favourable properties for the clustering method, although other Laplacians could also be used.

The Laplacian is characterized by being positive semi-definite, with the smallest eigenvalue equal to 0, whose multiplicity coincides with the number of connected components of the graph. We denote the eigenvalues by $0 = \lambda_1 \leq \dots \leq \lambda_N$, and the corresponding eigenvectors by $u_1, \dots, u_N \in \mathbb{R}^N$.

3.1.2 Algorithm

We present the spectral clustering method in several steps, assuming that the input is the $N \times N$ matrix W and the number of clusters is k .

1. Compute L_{rw} .
2. Compute the eigenvalues $\lambda_1, \dots, \lambda_N$ and corresponding eigenvectors $u_1, \dots, u_N$ of L_{rw} .
3. Let $U = (u_1 \mid \dots \mid u_k)$ be the matrix containing the first k eigenvectors as columns. Denote its N rows by vectors $y_i = (U_{i1}, \dots, U_{ik}) \in \mathbb{R}^k$.
4. Apply the k -means clustering algorithm to the set of vectors $\{y_1, \dots, y_N\}$ to obtain clusters $C_1, \dots, C_k \subseteq \{1, \dots, N\}$.
5. Define the clustered weight matrix w by

$$w_{ij} = \frac{1}{|C_i||C_j|} \sum_{l \in C_i} \sum_{s \in C_j} W_{ls}, \quad i \neq j, \quad (6)$$

and set $w_{ii} = 0$, where $|\cdot|$ denotes the cardinality of a set.

The algorithm identifies each node i of the graph with a vector $y_i \in \mathbb{R}^k$. In this Euclidean space, the k -means clustering algorithm is applied to the vectors y_i , and the resulting clustering is interpreted as a clustering of the nodes of the graph. Further details about k -means method are provided in the Appendix A.

Observe that, in the clustering algorithm, we considered the k eigenvectors of the Laplacian L_{rw} corresponding to the smallest eigenvalues. Since the Laplacian is defined in terms of the difference between the node degrees and the structural connectivity matrix, the algorithm indirectly preserves the directions associated, in some sense, with the dominant eigenvalues of the connectivity matrix, as previously introduced.

3.1.3 Clustering Quality

In the spectral clustering method described above, the number of clusters k is fixed beforehand. A general problem is to determine an optimal number of clusters. There exists a variety of methods that can be applied to address this problem.

In our study, we consider the *eigengap heuristic* as a tool specifically designed for spectral clustering, the *Q-modularity* as a metric for graph clustering, and general clustering quality metrics such as the *Silhouette* width and the *Calinski–Harabasz* index. These clustering metrics are described in Appendix B.

3.2 Clustering of the Empirical Structural Connectivity

In this section, we apply the spectral clustering method to the empirical structural connectivity graph.

Let W denote the considered 90×90 empirical structural connectivity matrix. A first clustering indicator is the eigengap heuristic in the Laplacian spectrum, which we illustrate in Figure 3.

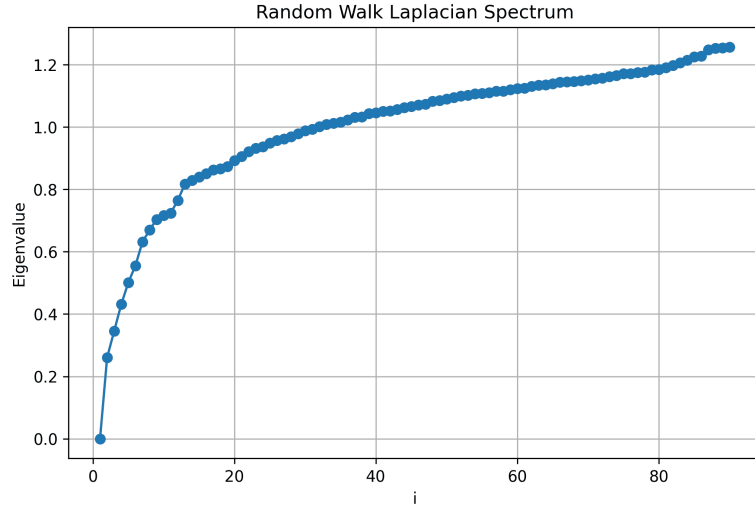


Figure 3: Eigenvalues of the random-walk Laplacian of the structural connectivity matrix W , ordered non-decreasingly.

In Figure 3 we observe that there is no clear eigengap for this connectivity, so we cannot apply the eigengap heuristic to estimate a suitable number of clusters. This indicates that the structural connectivity is dense, as expected for tractography data.

We proceed by applying the spectral clustering algorithm to W for different numbers of clusters and assess its quality using the indicators described above, in order to analyze whether there is any community structure in the underlying graph. The results of this study are summarized in Figure 4.

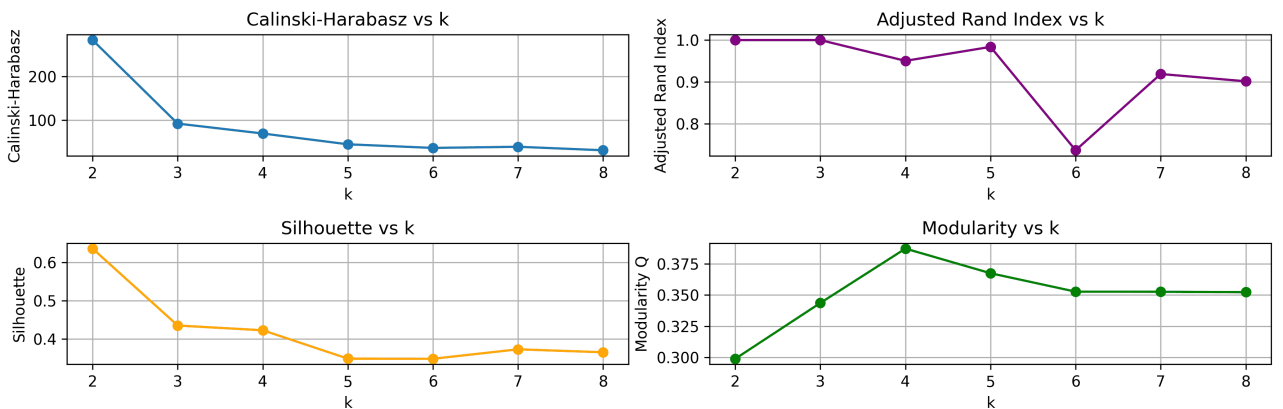


Figure 4: Clustering metrics for a low number of clusters, between 2 and 8. Larger numbers of clusters result in lower values for all the metrics.

Let us analyze the different plots in Figure 4. First, we observe that for the Calinski–Harabasz and Silhouette metrics, the values are large for $k = 2$, remain approximately stable for $k = 3$ and $k = 4$, and

then start decreasing. Except for $k = 2$, for which these metrics are less reliable, the overall low values suggest that there are no well-delimited clusters.

On the right, the ARI metric (see the final part of Appendix A) remains close to 1 up to $k = 5$, and then drops at $k = 6$. This indicates that for a small number of clusters, the obtained clustering is stable across different initializations, which may be an indicator of underlying community structure.

Perhaps the most interesting result is given by the Q -modularity metric, which is particularly relevant for graph clustering. The Q -modularity attains its maximum at $k = 4$, and then decreases for larger values of k . We observe that $Q(k = 4) \approx 0.4$, which suggests a strong clustering structure, since it lies in the range 0.3 - 0.7.

From this study, we conclude that there is no clear clustering structure in the connectivity graph, which is natural given that all 90 nodes of the original structural connectivity are expected to be interconnected. However, we identify an underlying community structure in the network, as indicated by a peak in the Q -modularity value at $k = 4$ clusters, which is one of the main indicators supporting this observation.

For this reason, in the rest of the project, we consider a 4-clustering of the structural connectivity, represented by a $N \times N$ matrix w with $N = 4$. Notably, this reduces the dimensionality of the system (1) from $90 \times 6 = 540$ to $4 \times 6 = 24$, which significantly reduces the computational cost of simulating the system.

An interactive visualization of the three-dimensional positions of the original 90 nodes and the resulting 4-clustering is available in the GitHub repository.

4. Linear Stability Analysis of Homogeneous Solutions

In this chapter, we perform a linear stability analysis to determine the stability of the homogeneous solutions under transverse perturbations in the large-scale brain model with clustered connectivity, depending on the external input parameter I_{ext}^E and the connectivity coupling strength ε .

We begin the chapter by presenting the homogeneous invariant manifold of the large-scale brain model and the associated solutions, namely homogeneous equilibrium points and periodic orbits. In the following section, we explain how to perform a stability analysis of these homogeneous solutions under transverse perturbations in the dynamical system (1). In particular, we describe how the Jacobian eigenvalues governing local stability are related to the structural connectivity spectrum through the dispersion relation (for equilibrium points) and the master stability function (for periodic orbits).

Later, we apply this theoretical framework to the clustered large-scale brain model to obtain the dispersion relation and master stability functions for different parameter pairs $(I_{\text{ext}}^E, \varepsilon)$. Next, we derive the bifurcation diagram of the clustered system, describing the different stability regions and the associated Jacobian eigenvalues as functions of the parameter pair $(I_{\text{ext}}^E, \varepsilon)$. We conclude the chapter by comparing the obtained results with those corresponding to the original structural connectivity matrix obtained in [1].

4.1 Homogeneous Invariant Manifold

In this section, we introduce the dynamics on the homogeneous invariant manifold.

Let $\mathbf{x}_j = (r_{E,j}, v_{E,j}, s_{E,j}, r_{I,j}, v_{I,j}, s_{I,j})$ denote the variables describing the dynamics of node j in system (1). If we impose identical dynamics across all nodes, that is,

$$\mathbf{x}_j = \mathbf{x} = (r_E, v_E, s_E, r_I, v_I, s_I), \quad j = 1, \dots, N,$$

then, as discussed in (3), all populations receive the same long-range input, $I = \varepsilon s_E$, which no longer depends on the dynamics of distinct populations.

Consequently, independently of the structural connectivity, the equations governing the evolution of the variables in (1) reduce to a system of six differential equations:

$$\begin{aligned} (E) \quad & \begin{cases} \tau_E \dot{r}_E = \frac{\Delta_E}{\pi \tau_E} + 2r_E v_E, \\ \tau_E \dot{v}_E = \bar{\eta}_E + v_E^2 - (\tau_E \pi r_E)^2 + I_{\text{ext}}^E + \tau_E ((J_{EE} + \varepsilon)s_E - J_{EI}s_I), \\ \tau_{s_E} \dot{s}_E = -s_E + r_E \end{cases} \\ (I) \quad & \begin{cases} \tau_I \dot{r}_I = \frac{\Delta_I}{\pi \tau_I} + 2r_I v_I, \\ \tau_I \dot{v}_I = \bar{\eta}_I + v_I^2 - (\tau_I \pi r_I)^2 + I_{\text{ext}}^I + \tau_I ((J_{IE} + \varepsilon)s_E - J_{II}s_I), \\ \tau_{s_I} \dot{s}_I = -s_I + r_I \end{cases} \end{aligned} \quad (7)$$

This reduced system describes the dynamics on the *homogeneous invariant manifold* of system (1).

The homogeneous system can be integrated numerically. We focus on the study of stable equilibrium points and periodic orbit solutions of the homogeneous system for different parameter selections and a null initial condition (i.e., by studying the ω -limit of the zero vector). Such regions can be detected by integrating the system from the null initial condition for a sufficiently long time and then identifying either convergence to a constant value for all variables (equilibrium point) or oscillatory behaviour (periodic orbit).

4.2 Linear Stability Analysis of Homogeneous Solutions

In this part, we present the linear stability analysis of the homogeneous fixed points and limit cycles of the full system (1). We apply a technique based on projecting perturbations onto a suitable eigenbasis, which decouples the high-dimensional stability problem into simpler lower-dimensional components associated with different eigenmodes. We then derive the dispersion relations associated with equilibrium points and the master stability functions associated with periodic orbits.

4.2.1 Structural Connectivity Spectrum

The aforementioned eigenbasis is formed by the eigenvectors of the structural connectivity matrix $\tilde{w}$. Note that, by the Gershgorin circle theorem [8], each eigenvalue Λ of $\tilde{w}$ satisfies the inequality

$$|\Lambda - \tilde{w}_{ii}| \leq \sum_{j \neq i} |\tilde{w}_{ij}|, \quad (8)$$

for at least one index $i \in \{1, \dots, N\}$. Since $\tilde{w}$ has a null diagonal and is row-normalized, the theorem implies that all eigenvalues are confined to the unit disk:

$$-1 \leq \Lambda_N \leq \dots \leq \Lambda_1 \leq 1. \quad (9)$$

We also note that, due to the row-normalization condition, the vector $\Phi_1 = (1, \dots, 1) \in \mathbb{R}^N$ is an eigenvector of $\tilde{w}$ with eigenvalue $\Lambda_1 = 1$, the largest possible value. By the Perron–Frobenius theorem [8], this is the unique eigenvector associated with the eigenvalue 1.

We will refer to the normalized eigenvectors Φ_α associated with the eigenvalues Λ_α of the structural connectivity $\tilde{w}$ as *eigenmodes*.

4.2.2 Perturbation Projection

Let $\mathbf{x}^{(0)} \in \mathbb{R}^6$ be either an equilibrium point (time-independent) or a periodic orbit (time-dependent) of the homogeneous system (7), and let

$$\mathbf{x}^{(0)} = (x^{(0)}, \dots, x^{(0)})^T \in (\mathbb{R}^6)^N \quad (10)$$

denote the corresponding homogeneous solution of the full system (1), which we write as

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{F}(\mathbf{x}(t)), \quad (11)$$

for the corresponding smooth vector field $\mathbf{F} : (\mathbb{R}^6)^N \rightarrow (\mathbb{R}^6)^N$. Consider an arbitrary small perturbation of $\mathbf{x}^{(0)}$,

$$\mathbf{x}_\delta(t) = \mathbf{x}^{(0)} + \delta\mathbf{x}(t), \quad \delta\mathbf{x} = ((\delta x)_1, \dots, (\delta x)_N) \in (\mathbb{R}^6)^N, \quad (12)$$

where $(\delta x)_j = (\delta x)_j(t)$ represents a perturbation acting on the j -th node. If we introduce $\mathbf{x}_\delta(t)$ into equation (11), and expand $\mathbf{F}(\mathbf{x}_\delta(t))$ up to first order in $\delta \mathbf{x}$, then, by equating both sides of the equation, we obtain that the linear evolution of the perturbation is governed by the first order variational equation

$$\frac{d}{dt}\delta \mathbf{x}(t) = \mathbf{J}(\mathbf{x}^{(0)})\delta \mathbf{x}(t), \quad (13)$$

where $\mathbf{J}$ is the full $6N \times 6N$ Jacobian matrix of the network system (1), evaluated at $\mathbf{x}^{(0)}$. The Jacobian matrix of the network can be decomposed into a term containing the Jacobian of the uncoupled system at each node and a coupling term:

$$\mathbf{J}(\mathbf{x}^{(0)}) = \mathbf{I}_N \otimes J(x^{(0)}) + \varepsilon \tilde{w} \otimes K, \quad (14)$$

where $\otimes$ denotes the Kronecker product, J is the 6×6 Jacobian matrix of the homogeneous system (7) with $\varepsilon = 0$, $\mathbf{I}_N$ is the $N \times N$ identity matrix, and $K = (k_{ij})$ is the 6×6 matrix whose entries are given by

$$k_{ij} = \begin{cases} 1 & \text{if } i \in \{2, 5\}, j = 3, \\ 0 & \text{otherwise.} \end{cases} \quad (15)$$

Note that the Jacobian $J(x^{(0)})$ can be computed analytically from (7) if $x^{(0)}$ is known.

We now express the perturbation in the connectivity eigenbasis, that is, in terms of the eigenmodes Φ_α :

$$\delta \mathbf{x}(t) = \sum_{\alpha=1}^N \Phi_\alpha \otimes \delta u_\alpha(t), \quad (16)$$

for some coordinates $\delta u_\alpha(t) \in \mathbb{R}^6$.

Substituting expressions (14) and (16) into system (13), we decouple it into N equations:

$$\frac{d}{dt}\delta u_\alpha(t) = \mathcal{J}(x^{(0)}, \Lambda_\alpha) \delta u_\alpha(t), \quad \alpha = 1, \dots, N, \quad (17)$$

where

$$\mathcal{J}(x^{(0)}, \Lambda_\alpha) := J(x^{(0)}) + \varepsilon \Lambda_\alpha K, \quad (18)$$

is a family of 6×6 Jacobian matrices depending on the homogeneous state $x^{(0)}$ and on the eigenvalues Λ_α of the structural connectivity matrix.

Since the family of Jacobians $\{\mathcal{J}(x^{(0)}, \Lambda_\alpha)\}_{\alpha=1}^N$ is obtained through a change of basis of the original Jacobian $\mathbf{J}(\mathbf{x}^{(0)})$, their spectra coincide. Therefore, the stability analysis of the homogeneous solution $\mathbf{x}^{(0)}$ can be carried out by studying the eigenvalues of the decoupled Jacobians $\mathcal{J}(x^{(0)}, \Lambda_\alpha)$ for each eigenmode $\alpha = 1, \dots, N$.

4.2.3 Dispersion Relation

When we assume $x^{(0)}$ to be an equilibrium point of the homogeneous system (7), the Jacobians $\mathcal{J}(x^{(0)}, \Lambda_\alpha)$ are time-independent, and the decoupled systems (17) become homogeneous linear differential equations with constant coefficients. In this case, the stability of the equilibrium point $\mathbf{x}^{(0)}$ of the network system (1) is determined by the eigenvalues of the Jacobians $\mathcal{J}(x^{(0)}, \Lambda_\alpha)$.

More precisely, the linear stability analysis based on the eigenmode decomposition depends on the largest exponential growth rate of perturbations acting along each eigenmode α ,

$$\mu_{\max}^{(\alpha)} := \max_{1 \leq j \leq 6} \Re(\mu_j^{(\alpha)}), \quad (19)$$

where $\Re$ denotes the real part, and $\mu_j^{(\alpha)}$ the j -th eigenvalue of $\mathcal{J}(x^{(0)}, \Lambda_\alpha)$. We observe that, for a given equilibrium point and fixed ε , expression (18) implies that $\mu_{\max}^{(\alpha)}$ depends only on the connectivity eigenvalue Λ_α :

$$\mu_{\max}^{(\alpha)} = \mathcal{F}(\Lambda_\alpha), \quad (20)$$

where the function $\mathcal{F}$ is known as the *dispersion relation*. A positive value of $\mu_{\max}^{(\alpha)}$ implies instability along the direction of the α -th eigenmode, whereas a negative value indicates stability along that direction. If all $\mu_{\max}^{(\alpha)} < 0$, then small heterogeneous perturbations of the homogeneous equilibrium point decay exponentially.

4.2.4 Master Stability Function

When we assume $x^{(0)}(t)$ to be a T -periodic solution of the homogeneous system (7), the Jacobians $\mathcal{J}(x^{(0)}(t), \Lambda_\alpha)$ are T -periodic, and the decoupled systems (17) become homogeneous T -periodic linear differential equations. In this case, the stability of the periodic orbit $x^{(0)}(t)$ of the network system (1) can be studied through the stability as a fixed point of the T -flow map.

Let $x^{(0)}(0) = x_0^{(0)}$ and write $x^{(0)}(t) = x^{(0)}(t, x_0^{(0)})$ to make the dependence on the initial condition explicit. Then, $x_0^{(0)}$ is a fixed point of the T -flow map

$$\mathbf{G}(\mathbf{x}) := \mathbf{x}^{(0)}(T, \mathbf{x}), \quad (21)$$

and its stability depends on the eigenvalues of

$$D_{\mathbf{x}}\mathbf{G}(x_0^{(0)}) = D_{\mathbf{x}}x^{(0)}(T, x_0^{(0)}) = D_{\mathbf{x}}x^{(0)}(T), \quad (22)$$

which is the monodromy matrix associated with the first-order variational equation. Note that $D_{\mathbf{x}}x^{(0)}(t)$ is the principal fundamental matrix of the first-order variational equation (13), and the monodromy matrix is obtained by integrating the system over one period T .

The eigenvalues of the monodromy matrix are known as the Floquet multipliers, denoted by ρ , and are all non-zero (by Liouville's formula). They can be expressed as $\rho = e^{\mu T}$, where μ are called Floquet exponents. Floquet exponents are not uniquely determined, but their real parts $\Re(\mu)$ are.

The stability criterion states that if there exists a Floquet multiplier such that $|\rho| > 1$, or equivalently a Floquet exponent satisfying $\Re(\mu) > 0$, then the periodic orbit is unstable. Conversely, if all Floquet multipliers satisfy $|\rho| < 1$, or equivalently all Floquet exponents satisfy $\Re(\mu) < 0$, then the periodic orbit is stable.

Under the projection of perturbations onto the eigenmodes, the variational equations reduce to (17), and the corresponding Floquet multipliers are denoted by $\{\rho_j^{(\alpha)}\}_{j=1}^6$, where α indexes the eigenmode.

The stability of the periodic orbit along each eigenmode direction is determined by

$$\mu_{\max}^{(\alpha)} := \max_{1 \leq j \leq 6} \Re(\mu_j^{(\alpha)}) = \max_{1 \leq j \leq 6} \Re\left(\frac{1}{T} \log \rho_j^{(\alpha)}\right), \quad (23)$$

where $\log$ denotes the complex logarithm and $\rho_j^{(\alpha)}$ is the j -th Floquet multiplier associated with $\mathcal{J}(x^{(0)}, \Lambda_\alpha)$. Similarly to the dispersion relation (20), we obtain a functional relation with the connectivity eigenvalues Λ_α , known as the *Master stability function*. The stability analysis is then analogous to that of the dispersion relation.

4.3 Computation of the Dispersion Relations and Master Stability Functions for the Clustered System

In this section, we consider pairs of free parameters $(I_{\text{ext}}^E, \varepsilon)$, compute the associated equilibrium point or periodic orbit, and derive the corresponding dispersion relation or master stability function for the large-scale brain model with clustered connectivity.

The conceptual workflow of the implementation included in the GitHub repository for this computation is as follows:

1. Consider a pair parameter values $(I_{\text{ext}}^E, \varepsilon)$.
2. Compute the ω -limit of the null vector in the homogeneous system, and classify it as an equilibrium point, a periodic orbit, or chaotic behaviour if neither occurs. In the periodic orbit case, also compute its period.
3. Proceed as follows:
 - If the ω -limit is an equilibrium point, obtain the dispersion relation by computing the eigenvalues of the Jacobians (18) for each eigenmode.
 - If the ω -limit is a periodic orbit, obtain the master stability function by computing the Floquet exponents, i.e., the eigenvalues of the Monodromy matrices obtained by integrating the variational equations (17) over one period for each eigenmode.

We include several dispersion relations and master stability functions corresponding to different parameter pairs in Figures 5 and 6.

Note that, in both cases, for each parameter pair, a different curve is shown, obtained over a continuous range of possible eigenvalues of the connectivity matrix. The circles represent the actual pairs $(\Lambda_\alpha, \mu_{\max}^{(\alpha)})$ for each eigenmode α of the considered connectivity matrix $\tilde{w}$. The connectivity eigenvalues Λ_α are the same in each representation, whereas the values $\mu_{\max}^{(\alpha)}$ vary for the different parameter pairs.

Those curves with some eigenmode satisfying $\mu_{\max}^{(\alpha)} > 0$ correspond to parameter pairs for which the homogeneous solution is unstable for the network system (1). On the contrary, if there is no eigenmode with $\mu_{\max}^{(\alpha)} > 0$, the homogeneous solution is stable for the network system.

For the homogeneous equilibrium points in Figure 5, we observe, for instance, in the case $\varepsilon = 8$, that the curves $I_{\text{ext}}^E = 16$ and $I_{\text{ext}}^E = 25$ have three unstable eigenmode directions for the clustered connectivity. On the contrary, the curves $I_{\text{ext}}^E = 1$ and $I_{\text{ext}}^E = 3$ indicate that the corresponding homogeneous solutions are stable for the network system.

Similarly, for the homogeneous periodic orbits in Figure 5, for example in the $\varepsilon = 9$ case, we observe that the curve $I_{\text{ext}}^E = 12$ has unstable eigenmodes, whereas curves such as $I_{\text{ext}}^E = 6$ correspond to stable solutions.

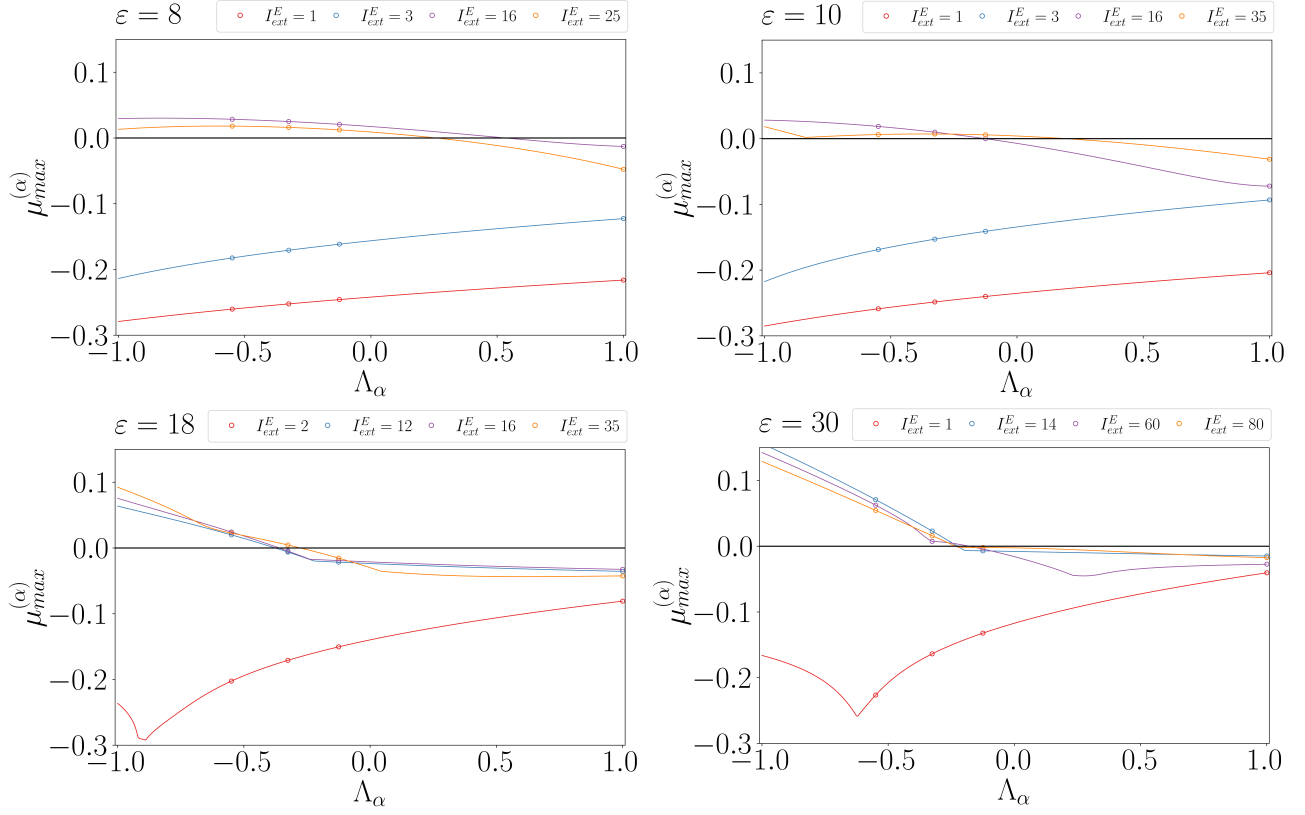


Figure 5: Dispersion relations for different ε values and I_{ext}^E .

We note that the master stability function always has a Floquet exponent equal to zero corresponding to the largest connectivity eigenvalue, $\Lambda_1 = 1$, independently of the parameter pair. Indeed, the eigenvector associated with this eigenvalue, $\Phi_1 = (1, \dots, 1) \in \mathbb{R}^N$, corresponds to a homogeneous perturbation tangent to the homogeneous periodic orbit. Therefore, the associated Floquet multiplier is equal to one, and the corresponding Floquet exponent is equal to zero.

Similarly, for the equilibrium points, the Jacobian eigenvalue associated with this same eigenmode is always negative, as can be seen in the dispersion relation plots.

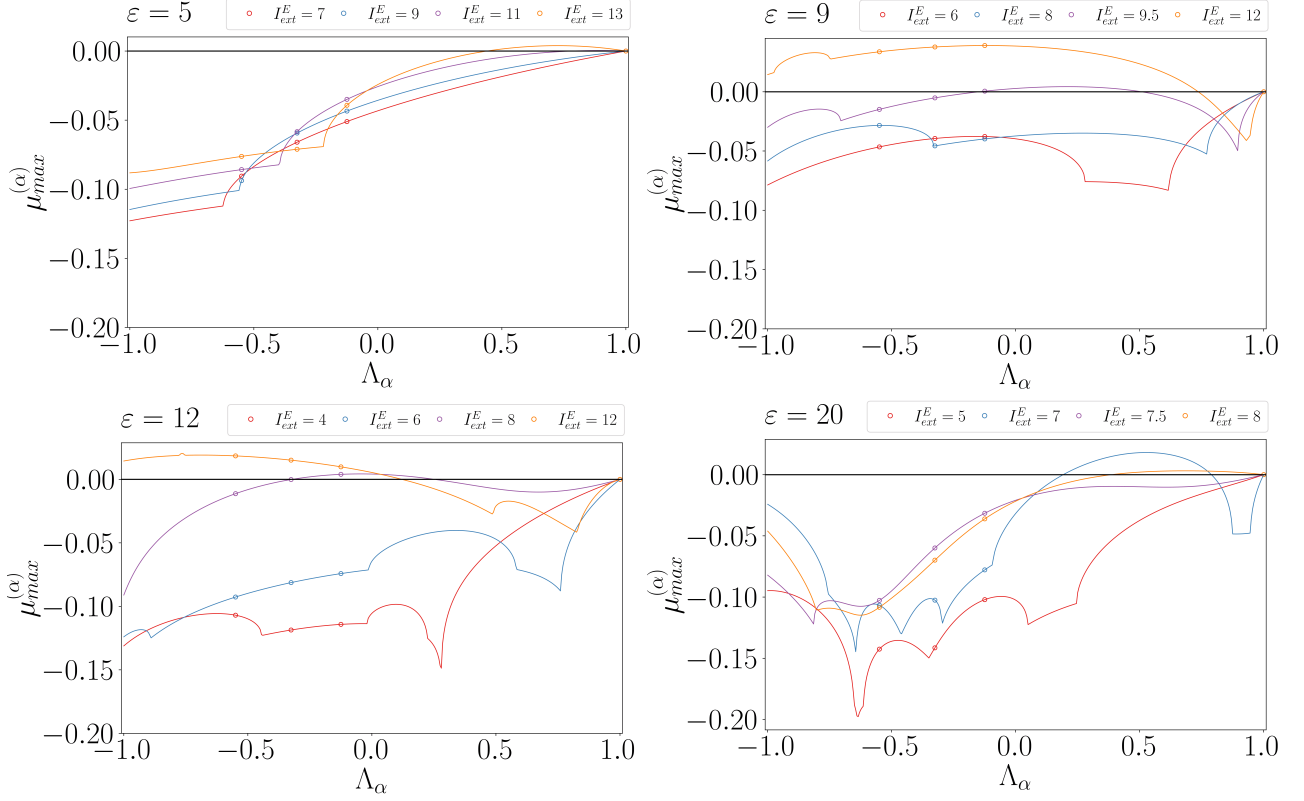


Figure 6: Master stability functions for different ε values and I_{ext}^E .

4.4 Bifurcation Diagram Computation

In this section, we show the bifurcation diagrams obtained by computing the dispersion relations and master stability functions over a grid of parameter values $(I_{\text{ext}}^E, \varepsilon)$.

Following the procedure described in the previous part over a grid in the range $[0, 16] \times [0, 35]$, we obtain the bifurcation diagrams shown in Figures 7 and 8. Let us describe Figure 7 in detail and then discuss the differences with Figure 8.

First, in Figure 7 we observe that, given a pair $(I_{\text{ext}}^E, \varepsilon)$ in the parameter grid, four main types of dots can be plotted. Dots in the colour region represented by the upper horizontal colour bar denote parameter pairs for which the homogeneous solution is a periodic orbit, and the corresponding colour indicates the value of the largest Floquet exponent across the different eigenmodes. Similarly, dots in the colour region represented by the lower horizontal colour bar denote parameter pairs for which the homogeneous solution is an equilibrium point, and the corresponding colour indicates the value of the largest Jacobian eigenvalue across the different eigenmodes.

We also observe grey dots, corresponding to parameter pairs for which neither a periodic orbit nor an equilibrium point was identified, and which are therefore classified as chaotic regions. We briefly discuss this behaviour in the following section. Finally, black dots correspond to points classified as equilibrium points or periodic orbits, but for which some numerical inconsistency occurred during the computations.

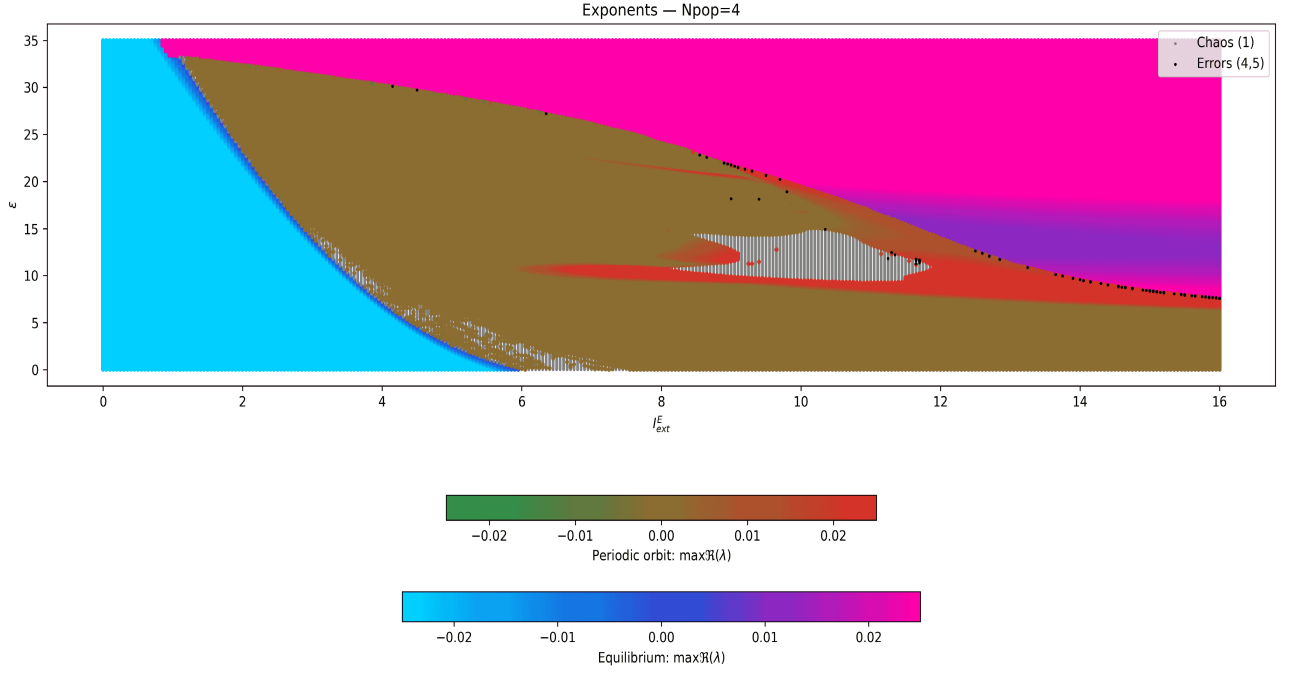


Figure 7: Bifurcation diagram complemented with the corresponding largest exponent over the eigenmodes of the clustered structural connectivity $\tilde{w}$.

We can distinguish three main regions according to the type of solution found. The brown and red lobular regions correspond to periodic orbits, whereas the blue, pink, and purple regions outside the lobe correspond to equilibrium points. In turn, the grey closed region inside the periodic-orbit lobe corresponds to chaotic behaviour.

We also observe that some chaotic points appear near the boundary of the periodic-orbit region, and that some numerical errors occur close to these boundaries as well. This may be due to the fact that the classifier struggles in transition regions separating different types of dynamical behaviour.

Let us focus on the periodic-orbit region. We observe that this region is primarily filled with parameter pairs whose maximum Floquet exponent is equal to zero (brown region), corresponding to stable homogeneous periodic orbits. There are also some regions close to the chaotic region with a positive exponent (red region), corresponding to unstable periodic orbits. There are no regions with negative values of the maximum exponent, since, as previously discussed, zero is always a Floquet exponent.

If we now examine the equilibrium-point region, we observe that the region to the left of the periodic-orbit regime (cyan region) is filled with stable equilibrium points, whereas the region to the right (pink–purple region) contains unstable periodic points. A nearly horizontal purple band of lower positive exponent is visible, approximately in the range $\varepsilon \sim 10$ –15.

Figure 8 is similar to the previous one, but instead of showing the largest exponent, it displays the number of positive exponents across the different eigenmodes, that is, the number of unstable eigenmode directions.

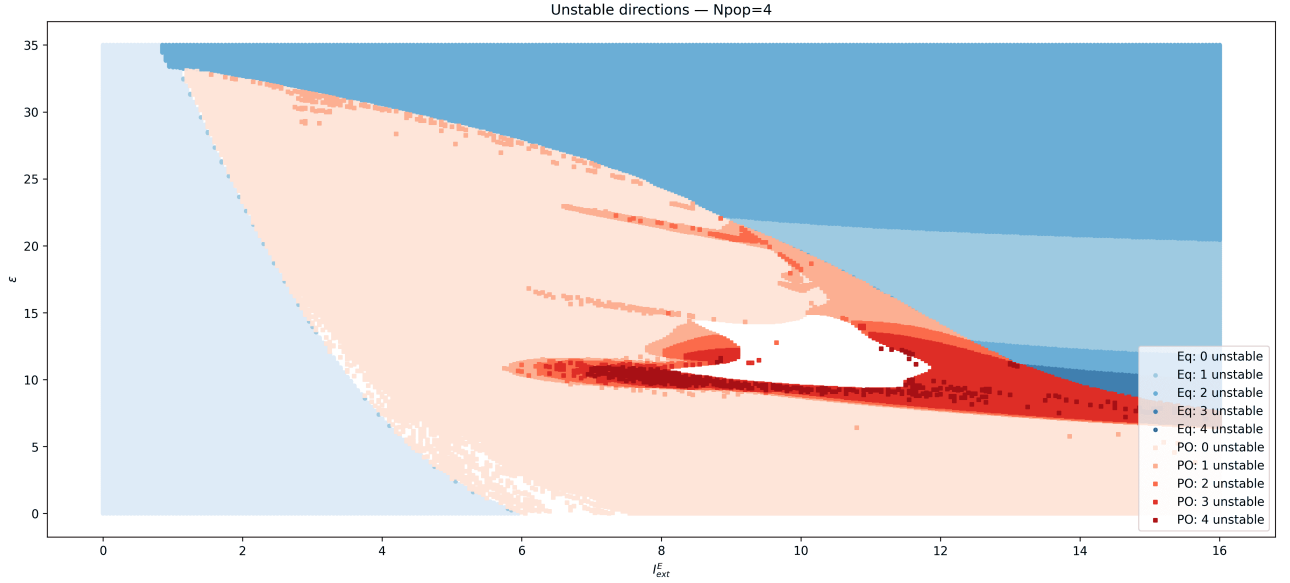


Figure 8: Bifurcation diagram complemented with the corresponding number of unstable directions (i.e., positive exponents) across the eigenmodes of the clustered structural connectivity $\tilde{W}$.

In the periodic-orbit regime, we observe a large lobule below the chaotic region in which up to four unstable directions are present. As we move above the chaotic region, close to the right boundary, the number of unstable directions decreases, and two secondary, elongated lobules emerge.

In the unstable equilibrium-point regime, we observe nearly horizontal bands with different numbers of unstable directions. As we move upwards (increasing ε), the bands correspond to 3, 2, 1, and again 2 unstable directions.

4.5 Comparison with the Original Structural Connectivity

We conclude the chapter by comparing the results obtained for the clustered 4×4 connectivity matrix, with those obtained for the original 90×90 structural connectivity matrix in [1].

We can observe some similarities and differences between the results for the clustered connectivity in Figures 7 and 8, and the results for the original connectivity in Figure 9.

If we compare the periodic-orbit regime, we note that the large unstable lobule near the chaotic region appears in both cases, for the clustered and original connectivities. In both cases, the number of unstable directions in this region is close to the number of nodes.

For the original connectivity, we observe several secondary elongated unstable lobules, but with few unstable directions. The clustered connectivity is able to preserve two of these lobules, but since the number of possible unstable directions is much lower, we lose complexity in this structure.

The stable equilibrium-point regime to the left of the periodic-orbit regime is shared by both connectiv-

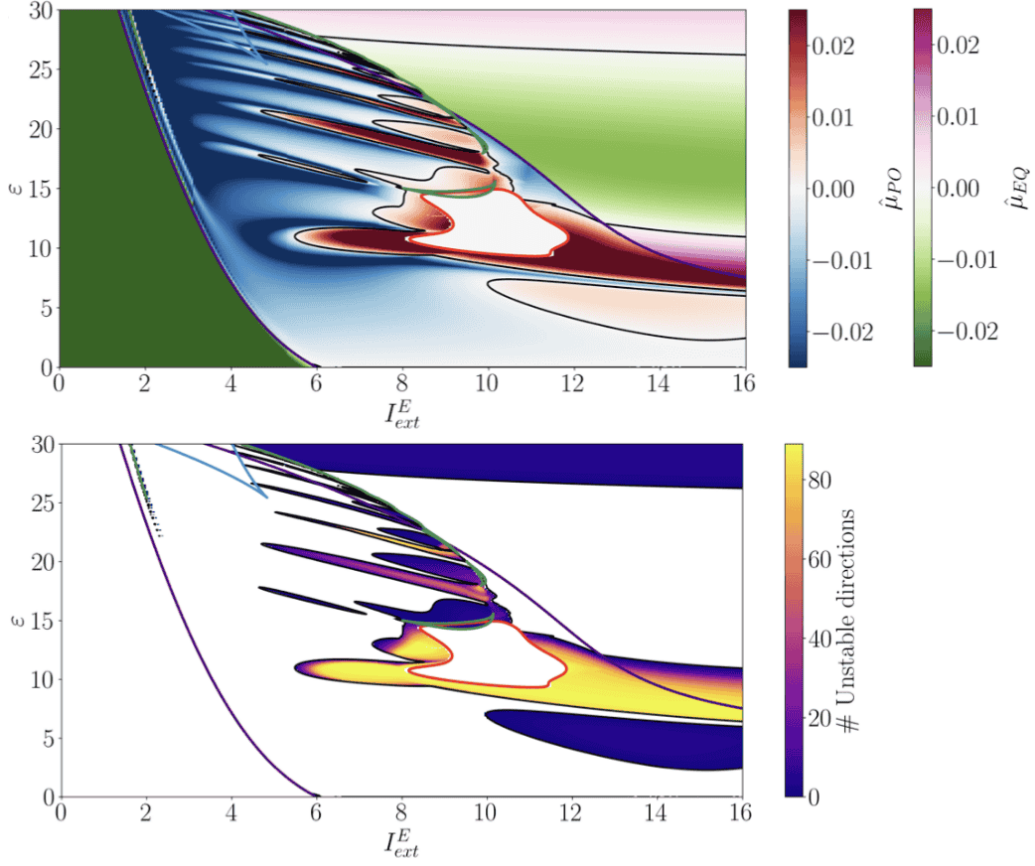


Figure 9: Bifurcation diagrams extracted from [1]. The upper panel shows the largest exponent over the eigenmodes of the original structural connectivity $\tilde{W}$, excluding the exponent corresponding to the eigenmode $\Lambda_\alpha = 1$. The lower panel shows the number of unstable directions.

ities. However, the equilibrium-point regime to the right of the periodic-orbit regime exhibits a significant difference.

First, we note that, close to the lower part of the periodic-orbit regime boundary, both connectivities have a region with a large number of unstable directions, which decreases as we move away (increasing ε).

For the clustered connectivity, as ε increases, the number of unstable directions decreases until reaching a region with a single unstable direction, and then increases again. In contrast, for the original connectivity, the number of unstable directions gradually decreases until reaching a stable equilibrium-point region, not present in the clustered case. As ε keeps increasing, an unstable region is reached again, with a small number of unstable directions.

It is worth noting that the differences and similarities in the bifurcation diagrams for the clustered and original connectivities depend only on the eigenvalues of the connectivity matrix, as discussed in the dispersion relation and master stability function sections. Thus, we may compare the eigenvalues obtained for the clustered connectivity with those of the original connectivity.

Figure 10 compares the spectral differences between the two connectivities and their influence on some illustrative dispersion relations.

Through these dispersion relations, we observe that the two lowest eigenvalues of the clustered connec-

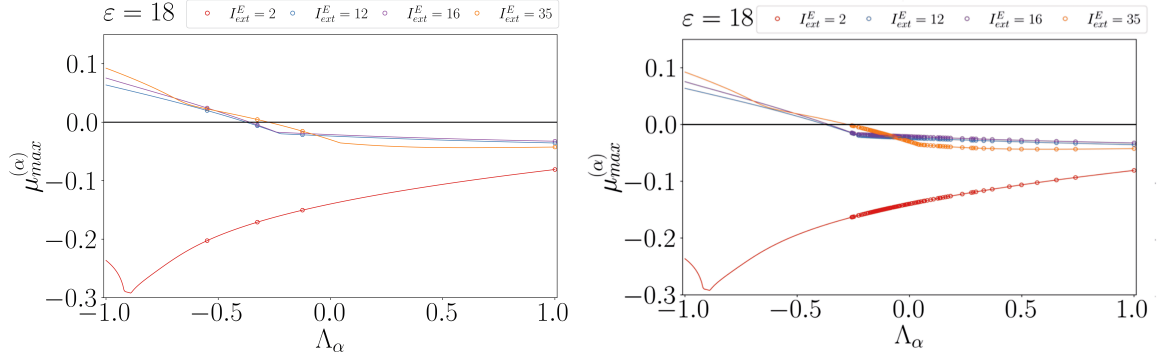


Figure 10: Dispersion relations for the clustered connectivity (left) and the original connectivity (right).

tivity are lower than all the eigenvalues of the original connectivity. This causes the significant difference discussed earlier. For instance, if we consider the dispersion relations in Figure 10 corresponding to $I_{ext}^E = 12$ or 16, the clustered connectivity has an unstable direction, whereas the original connectivity has no unstable directions, which is the behaviour previously described in this range of parameters.

We conclude that the clustered connectivity is capable of preserving the stability analysis of the original connectivity to a large extent, for instance by preserving the regions with a large number of unstable directions. However, some parameter regions may exhibit important stability changes due to the clusterization, such as the equilibrium-point region for large I_{ext}^E and ε discussed above.

5. DeepONet Predictions for Phase-Amplitude Dynamics

In this chapter, we introduce the DeepONet neural network architecture and its application to predicting general solutions of initial value problems in dynamical systems depending on parameters. We apply this methodology to approximate the solution of the phase-amplitude model, a low-dimensional system describing the local dynamics around a stable homogeneous periodic orbit of the large-scale brain model.

We begin the chapter by motivating the problem. Next, we present the phase-amplitude coupling dynamical system and how it approximates the dynamics around a stable homogeneous limit cycle of (1). Then, we introduce the DeepONet neural network and how it can be used to approximate solutions of initial value problems for parameter-dependent systems, such as the phase-amplitude system. We describe the different stages of the DeepONet methodology—architecture design, training, and prediction—and how it can be adapted to our specific problem. Finally, we develop a software implementation of the methodology and illustrate its prediction accuracy by comparing it with numerically integrated trajectories, as well as its capability for solving the associated inverse problem.

5.1 Motivation of the Problem

In this section, we briefly motivate the consideration of the phase-amplitude dynamical system and the application of DeepONet.

We introduce the phase-amplitude coupling low-dimensional model for several reasons. Since the large-scale brain dynamical system is complex and high-dimensional, a natural introductory approach is to consider a reduced system that describes the dynamics in a localized region. Specifically, the phase-amplitude coupling system fixes all the parameters in Table 1 except for the coupling strength, and is therefore well-suited to explore this specific dependence.

As we will see next, this system depends on functions that can be approximated to some extent but are difficult to handle in practice. For this reason, we propose DeepONet as an alternative approach that relies on these functions during training but, after training, is able to predict solutions instantaneously without explicitly depending on them. Additionally, it offers capabilities that are limited for classical approaches, such as fast and direct predictions and the ability to infer trajectories from noisy data (inverse problem), as will be shown in the end of the chapter.

Moreover, DeepONets are non-trivial to implement for complex systems in practice; thus, a low-dimensional system such as the phase-amplitude model provides an ideal introductory case for studying and applying neural operator learning methods.

5.2 Phase-Amplitude Coupling Dynamics

In this section, we introduce the phase-amplitude coupling system as a reduced low-dimensional model for the neuronal dynamics (1) in a neighbourhood of a stable homogeneous periodic orbit.

Let I_{ext}^E be a parameter value for which the uncoupled homogeneous system, that is, homogeneous system (7) with $\varepsilon = 0$, admits a stable homogeneous periodic orbit. For instance, this corresponds to any pair $(I_{\text{ext}}^E, 0)$ lying in the brown region of Figure 7.

Let T denote its period and $\lambda < 0$ its largest non-zero Floquet exponent. We then consider the reduced dynamics of the coupled system (7) for $\varepsilon > 0$ restricted to the attracting manifold associated with the Floquet exponent λ . This manifold is referred to as a *slow attracting manifold*, since λ corresponds to the dominant non-zero decay rate.

We parameterize this slow attracting manifold using *phase–amplitude coordinates* (θ, σ) , where $\theta \in [0, T)$ and $\sigma \in \mathbb{R}$, such that the dynamics of these coordinates on the manifold are given by

$$\begin{cases} \dot{\theta} = 1 + \varepsilon Z(\theta, \sigma) K^{s_E}(\theta, \sigma), \\ \dot{\sigma} = \lambda \sigma + \varepsilon I(\theta, \sigma) K^{s_E}(\theta, \sigma), \end{cases} \quad (24)$$

for certain scalar functions $Z(\theta, \sigma)$ and $I(\theta, \sigma)$ known as phase and amplitude response curves, respectively, and a vector-valued function $K(\theta, \sigma)$ which is the parameterization of the slow manifold, with $K^{s_E}(\theta, \sigma)$ denoting its s_E component. These functions, which depend on the specific parameter selection (Table 1), can be written using Taylor expansions in σ as

$$Z(\theta, \sigma) = \sum_{n=0}^{\infty} Z_n(\theta) \sigma^n, \quad I(\theta, \sigma) = \sum_{n=0}^{\infty} I_n(\theta) \sigma^n, \quad K(\theta, \sigma) = \sum_{n=0}^{\infty} K_n(\theta) \sigma^n. \quad (25)$$

and are numerically approximated via Fourier expansions.

Further details on this dimensional reduction procedure and the computation of these functions can be found in [9].

5.3 DeepONet Estimation of Solutions to the Parametrized Initial Value Problem

In a general perspective, the phase–amplitude coupling system (24) is a particular example of an initial value problem (IVP) associated with a parameter-dependent ordinary differential equation. In this section, we introduce the DeepONet methodology for the approximation of solutions of parametrized initial value problems, as a first step towards our goal of predicting the solution of the phase–amplitude system.

Consider the initial value problem:

$$\begin{cases} \frac{d}{dt} \mathbf{x}(t) = \mathbf{F}(\mathbf{x}(t), \mu), \\ \mathbf{x}(0) = \mathbf{x}_0, \end{cases} \quad (26)$$

for a smooth vector field $\mathbf{F} : \mathbb{R}^n \times U \rightarrow \mathbb{R}^n$, where $U \subseteq \mathbb{R}^k$ is the parameter space, $\mu \in U$ is a k -dimensional parameter, and the initial conditions satisfy $\mathbf{x}_0 \in V \subseteq \mathbb{R}^n$, with V an n -dimensional space.

We want to estimate the general solution of this IVP; that is, a smooth function

$$\begin{aligned} \phi : U \times V \times [0, +\infty) &\longrightarrow \mathbb{R}^n \\ (\mu, \mathbf{x}_0, t) &\longmapsto \phi(\mu, \mathbf{x}_0)(t) \end{aligned} \quad (27)$$

which satisfies (26) for all parameters μ and initial conditions $\mathbf{x}_0$.

The method we explore in this work consists of using neural networks to estimate the general solution of the IVP (27). We propose a neural network architecture based on Deep Operator Networks (DeepONets), a method introduced in [10] for learning nonlinear operators.

DeepONets are able to accurately approximate nonlinear continuous functionals (mappings from a space of functions into the real numbers) and, more generally, nonlinear operators (mappings from a space of functions into another space of functions). This approach is based on the universal approximation theorem for operators [11], and is realized in practice through a concrete neural network architecture consisting of two subnetworks: one for encoding the input function (the branch net), and another for encoding the locations of the output function (the trunk net).

Our problem, however, is simpler than the one considered in the general DeepONet framework, in the sense that the function (27) is not an operator but instead depends on parameters and initial conditions. For this reason, we next present how the DeepONet architecture can be modified to capture the specific structure of our problem.

5.3.1 DeepONet Architecture Applied to Parametrized Functions

In what follows, we present how the DeepONet approximation is constructed through linear combinations of functions and parameters, and how it can be adapted to our specific problem.

Let

$$\begin{aligned}\phi : W \times [0, +\infty) &\longrightarrow \mathbb{R}^n \\ (\gamma, t) &\longmapsto \phi(\gamma)(t)\end{aligned}\tag{28}$$

be the parametrized function we want to approximate, where $W \subseteq \mathbb{R}^m$ is the m -dimensional space of *input parameters* $\gamma \in W$.

We propose an estimation function with the structure

$$\hat{\phi}(\gamma)(t) = \sum_{i=1}^p T_i(t) \mathbf{B}_i(\gamma)\tag{29}$$

where $p \in \mathbb{N}$ is called the *basis dimension*, the one-dimensional functions $T_i(t)$ are called the *trunk functions*, and the n -dimensional functions $\mathbf{B}_i(\gamma)$ are called the *branch functions*. We next make these functions explicit.

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous non-polynomial function, which we refer to as the *activation function*. We then consider trunk functions of the form

$$T_i(t) = \sigma(w_i t + \eta_i),\tag{30}$$

where $w_i, \eta_i \in \mathbb{R}$ are real parameters, and branch functions $\mathbf{B}_i = (B_i^{(1)}, \dots, B_i^{(n)})$ with components

$$B_i^{(\alpha)}(\gamma) = \sigma \left(\sum_{j=1}^m \xi_{ij}^{(\alpha)} \gamma_j + \theta_i^{(\alpha)} \right),\tag{31}$$

where $\xi_{ij}^{(\alpha)}, \theta_i^{(\alpha)} \in \mathbb{R}$ are also real parameters.

In practice, this approximation is implemented through a neural network architecture referred to as the (unstacked) *DeepONet*, which consists of two shallow dense neural networks layers, namely the trunk and branch networks. This structure is illustrated in Figure 11.

Each shallow dense neural network consists of an input layer, represented by a list of real values $x = (x_1, \dots, x_n)$, and an output layer composed of *neurons* (also called perceptrons), which produce a list of real values $y = (y_1, \dots, y_m)$. Each input x_j is connected to an output neuron y_i through a weight ω_{ij} . A neuron i performs the following mathematical operation:

$$y_i = \sigma \left(\sum_{j=1}^n \omega_{ij} x_j + b_i \right) \quad (32)$$

where b_i is the bias associated with neuron i , and σ is the neuron activation function. The collection of weights and biases,

$$\{\omega_{ij}, b_i \in \mathbb{R} : i \in \{1, \dots, m\}, j \in \{1, \dots, n\}\},$$

is known as the set of *network parameters*.

In the DeepONet architecture, we consider a trunk shallow dense layer with weights w_i and biases η_i , and a branch shallow dense layer with weights $\xi_{ij}^{(\alpha)}$ and biases $\theta_i^{(\alpha)}$. In later sections, we describe how these network parameters are estimated through what is known as the training process.

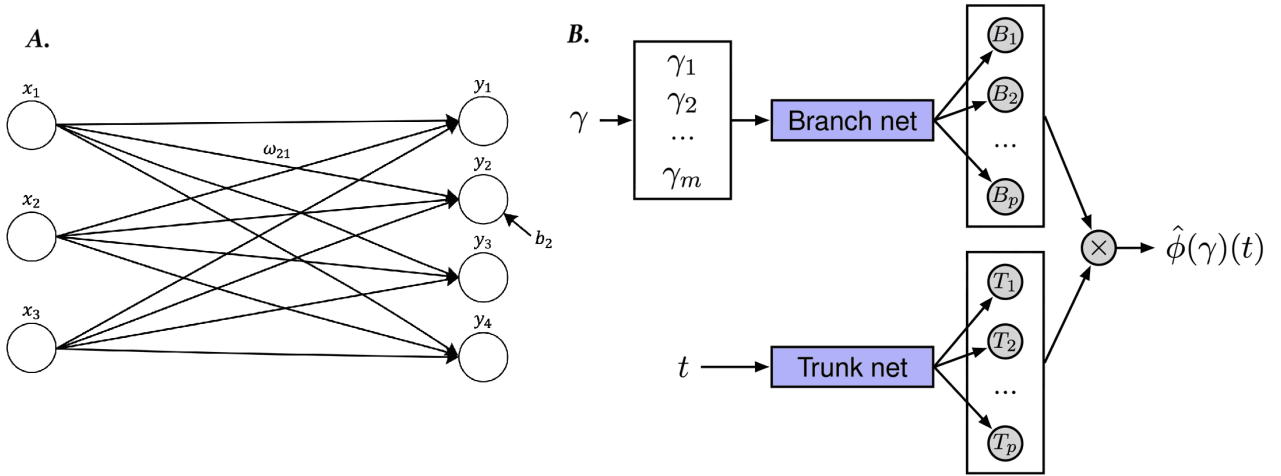


Figure 11: A. Illustration of a simple dense layer. B. Illustration of the (unstacked) DeepONet architecture for parametrized function approximation. Adapted from [10].

5.3.2 DeepONet Architecture Design Choices

In this part, we describe the design choices adopted for the DeepONet architecture, aimed at improving approximation performance and adapting it to our problem.

First, we note that although shallow neural networks are theoretically capable of approximating the target function, they exhibit several limitations in practice. Indeed, the universal approximation theorem guarantees a small approximation error for sufficiently large networks (i.e. a large basis dimension p), but it does not address how to optimize the training process. Additionally, it does not consider how to improve

the generalization performance of the network, which refers to its ability to perform well on input data not considered during training.

In practice, it is preferable to use deep neural networks (DNNs) instead of shallow ones. DNNs have a structure similar to that of shallow neural networks but include intermediate hidden layers of neurons between the input and output layers. DNNs can approximate target functions with higher parameter efficiency than shallow networks, meaning that comparable accuracy can often be achieved with fewer trainable parameters. This can reduce the computational cost of training and may facilitate the optimization process in practice. Additionally, the hierarchical structure of DNNs leads to greater expressivity, as relevant features of the target function can be learned in a compositional way. This feature learning may improve the generalization performance of the network, provided that overfitting is avoided.

By incorporating DNNs, we increase both the number of learnable parameters and the number of hyperparameters of the model. Initially, the basis dimension p was the only hyperparameter; we now additionally consider the dimensions of the hidden layers of the DNNs (i.e. the number of neurons per layer), namely the *hidden dimensions*; the dimensions of the output layers of each DNN, referred to as the *latent dimensions*; and the number of layers per DNN, which we refer to as the *depths* of the networks.

Secondly, we observe that in the DeepONet architecture, two distinct subnetworks are considered, each receiving a different type of input. A naive approach would consist of treating all inputs equally and using a single neural network. However, since the input parameter γ and the scalar input t have different roles in the considered problem, we incorporate this prior structure into the architecture by using two separate branch and trunk subnetworks.

Similarly, in the IVP solution approximation (26), we have two different types of input parameters, namely the real parameters of the system μ and the initial conditions $\mathbf{x}_0$. Thus, it is convenient to use different DNNs for each type of input instead of a single DNN. Additionally, to allow the network to learn possible interactions between the parameters μ and the initial conditions $\mathbf{x}_0$, we incorporate a *merger network* that takes as input the concatenation of the outputs of the parameter and initial-condition DNNs, and outputs the branch functions $\mathbf{B}_i(\mu, \mathbf{x}_0)$. This architecture is illustrated in Figure 12.

A third aspect of the implementation concerns the type of input and the choice of activation function. To capture the nonlinearities of the target function, we use the hyperbolic tangent $\tanh$, which is approximately linear near zero and saturates as the input moves away from the origin. This type of activation is particularly effective when the inputs are appropriately normalized. Since our inputs μ , $\mathbf{x}_0$, and t take values in continuous domains that are not a priori bounded, a prior scaling is typically required before they are used as inputs of the network. The scaling of each input depends on the range of values to be explored.

A final modification of the DeepONet architecture is useful when we aim to capture oscillatory dynamics in (27). In such cases, this prior information can be incorporated by modifying the input to the trunk network: instead of a single scalar t , we consider a vector of *Fourier features*, obtained by concatenating pairs $(\sin(2\pi ft), \cos(2\pi ft))$ for different frequencies f in a set $F \subseteq \mathbb{R}$ (see Figure 12). The set F is treated as a collection of hyperparameters and is chosen depending on the expected oscillatory behaviour.

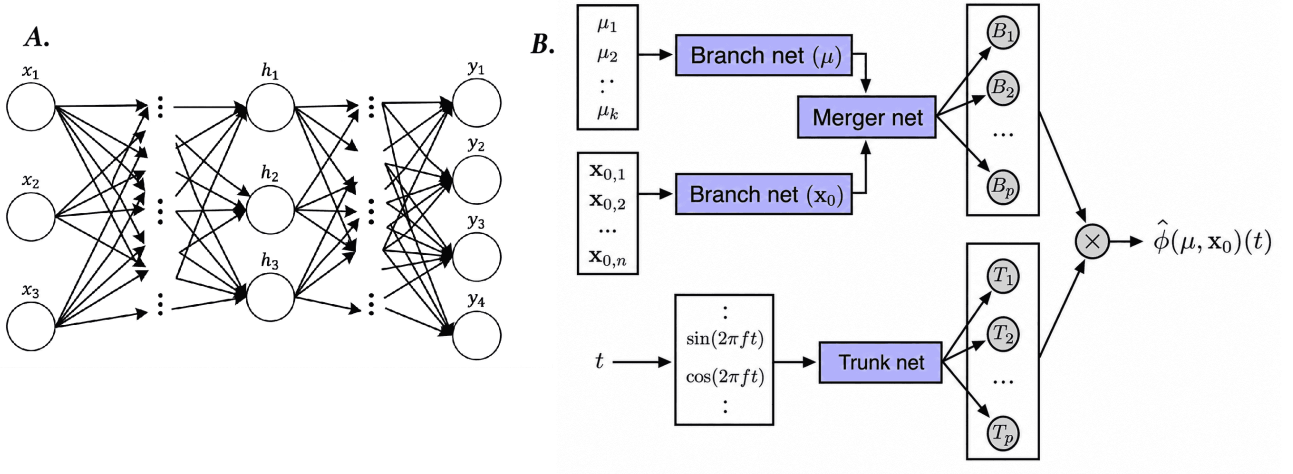


Figure 12: A. Illustration of a simple DNN. B. Illustration of the adapted DeepONet architecture for IVP oscillatory solution approximation.

5.4 DeepONet Training

In this section, we explain how the training process is carried out for the considered IVP solution estimation problem.

5.4.1 Loss Optimization

The general purpose of the training process is to learn to estimate the network parameters for which the function $\hat{\phi}$ (29) best approximates the target function ϕ (28), for a fixed choice of hyperparameters (that is, a fixed network architecture).

This is achieved by considering a *loss function*, denoted by $L(\hat{\phi})$, which quantifies how accurately $\hat{\phi}$ approximates ϕ . Then, if $\Theta \in \mathbb{R}^\ell$ denotes the DeepONet parameters, the optimal parameters Θ^* are defined as the solution of the optimization problem

$$\Theta^* = \arg \min_{\Theta \in \mathbb{R}^\ell} \underbrace{L(\hat{\phi}(\Theta))}_{L(\Theta)}. \quad (33)$$

In general, there is no guarantee that such a solution exists, nor that it can be found explicitly. Moreover, for DNN architectures, the loss function is typically non-convex with respect to the network parameters, and thus most optimization methods can converge to a local rather than global minimum. Nevertheless, optimization methods are often able to find parameters yielding sufficiently small loss values in practice.

Typically, for DNN architectures, this optimization problem is solved using a variant of the *gradient descent algorithm*. This algorithm first initializes the network parameters and then updates them iteratively via

$$\Theta_{k+1} = \Theta_k - \eta \nabla_{\Theta} L(\Theta_k), \quad (34)$$

where $\eta > 0$ is the *learning rate*, and $\nabla_{\Theta} L(\Theta_k)$ denotes the gradient of the loss function with respect to the parameters evaluated at iteration k .

The gradient of the loss function can be computed using backpropagation, which is essentially the repeated application of the chain rule from the output of the DeepONet back to its inputs. Therefore, the expression of the gradient depends on the specific loss function under consideration and on the architecture of the neural network.

In practice, modern deep learning frameworks use reverse-mode automatic differentiation and computational graphs to compute gradients [12]. Computational graphs represent the sequence of operations involved in the computation as a directed acyclic graph, and automatic differentiation exploits this structure to apply the chain rule efficiently and obtain the gradients.

Gradient descent algorithm generates a sequence of parameters that, under suitable conditions, may converge to a local minimum. The resulting limit point depends on the initial condition and the learning rate. This is because, at each iteration, the parameter Θ is updated in the direction of steepest local decrease of the loss function, given by the negative gradient with respect to Θ .

Nevertheless, there is no guarantee that the algorithm will converge to a global minimum (34), and it may instead become trapped in a local minimum or exhibit oscillatory behaviour. For this reason, there exists a wide variety of algorithms that modify the gradient descent loop to improve the training process [13].

Specifically, in this work we will consider the *Adam algorithm* [13] as a variant of gradient descent. This is an optimization method that computes adaptive learning rates at each iteration, combining first- and second-moment estimates of the gradients, which typically leads to faster convergence and improved stability compared to standard gradient descent algorithms.

In addition, we incorporate *gradient clipping* to improve numerical stability during training. More precisely, we bound the norm of the gradient at each iteration in order to prevent excessively large parameter updates in regions of steep curvature, while preserving the descent direction.

Finally, we initialize the network parameters using the *Xavier/Glorot initialization*, which is designed to preserve the variance of activations across layers at initialization. This helps avoid vanishing or exploding activations and typically leads to more stable and efficient training, especially when using symmetric activation functions such as tanh.

These training techniques are described in Appendix C.

5.4.2 Types of Loss Functions

In this part, we introduce the relevant loss functions for the considered problem.

Supervision Loss: A first possible loss function is the *supervision loss*. For the considered problem, this loss compares known trajectories of the IVP, $\phi(\mu, \mathbf{x}_0)(t)$, with the corresponding DeepONet approximation $\hat{\phi}(\mu, \mathbf{x}_0)(t)$:

$$L_{\text{superv}} = \left\| \hat{\phi}(\mu, \mathbf{x}_0)(t) - \phi(\mu, \mathbf{x}_0)(t) \right\|^2. \quad (35)$$

Let us consider a sample of numerical trajectories of the form

$$D = \{ \phi(\mu, \mathbf{x}_0)(t) : t \in D_{\text{time}} \}_{(\mu, \mathbf{x}_0) \in D_{\text{traj}}}, \quad (36)$$

where D_{traj} is a finite collection of parameter and initial condition pairs, and D_{time} is a finite collection of times, which may in general depend on each $(\mu', \mathbf{x}'_0) \in D_{\text{traj}}$. The collection D constitutes the *training dataset*.

The training process associated with the supervision loss consists, for each pair $(\mu', \mathbf{x}'_0) \in D_{\text{traj}}$ and each time $t' \in D_{\text{time}}$, of performing a parameter update of the DeepONet according to one iteration of the gradient descent algorithm applied to the supervision loss:

$$\Theta \leftarrow \Theta - \eta \Delta \Theta(g), \quad g = \nabla_{\Theta} L(\hat{\phi}(\Theta)), \quad \hat{\phi} = \hat{\phi}(\mu', \mathbf{x}'_0)(t'). \quad (37)$$

In practice, parameter updates are not performed for each individual triple $(\mu', \mathbf{x}'_0, t')$, but after considering the accumulated loss over a collection of such samples, called a *batch*. This approach is known as mini-batch gradient descent optimization. Typically, the training dataset is randomly partitioned into a certain number of batches, of a certain size S_{batch} , and a parameter update is performed after evaluating the average loss over each batch.

Moreover, it has been observed that repeating the training process using different random batch partitions of the same dataset improves the performance and generalization capability of the neural network. These repetitions are called *epochs*, and their total number is denoted by N_{epoch} .

IVP-Specific Losses: We now consider loss functions that are specific to the IVP solution estimation problem. We note that the IVP (26) imposes two necessary conditions on the solution: it must satisfy both the differential equation (ODE) and the initial condition (IC). These conditions can therefore be interpreted as different sources of loss, and the network can be trained to minimize them.

An *initial condition loss* can be defined as

$$L_{\text{IC}} = \left\| \hat{\phi}(\mu, \mathbf{x}_0)(0) - \mathbf{x}_0 \right\|^2. \quad (38)$$

Similarly, a *differential equation loss* is defined by

$$L_{\text{ODE}} = \left\| \frac{d}{dt} \hat{\phi}(\mu, \mathbf{x}_0)(t) - \mathbf{F}(\hat{\phi}(\mu, \mathbf{x}_0)(t), \mu) \right\|^2. \quad (39)$$

The previous loss is a pointwise residual evaluated at a specific time t . In practice, this loss is averaged over batches of sampled parameters, initial conditions, and time instances.

We note that the derivative of the neural network approximation with respect to t is computed via automatic differentiation, similarly to the gradient of the loss function. Moreover, when the considered dynamical system contains variables with significantly different scales, it is convenient to normalize the argument inside the norm of the ODE loss in order to balance the contribution of the different variables.

These types of losses are used in what are known as *physics-informed neural networks* (PINNs) [14], where the physical structure of the problem is explicitly incorporated into the training process. Other physics-informed loss functions may also be considered; for instance, losses associated with the variational equations of the system, which are relevant when one wants the approximate solution to satisfy higher-order differential relations.

5.5 DeepONet Prediction and the Inverse Problem

In this section, we describe the prediction capabilities of a DeepONet model trained to approximate solutions of IVPs, and how it can be used in the inverse problem setting.

Let us assume that the DeepONet has been trained. Then, the DeepONet can be used to predict the target function $\phi(\mu, \mathbf{x}_0)$ for a given parameter vector μ and initial condition $\mathbf{x}_0$. This is a straightforward process once the model is trained: we simply select a desired collection of time points P_{time} where the trajectory is to be evaluated, and for each $t \in P_{\text{time}}$, we feed the inputs $(\mu, \mathbf{x}_0, t)$ into the DeepONet to obtain the approximation $\hat{\phi}(\mu, \mathbf{x}_0)(t)$ (forward propagation).

However, the neural network can also be used to address the *inverse problem* (IP): given a trajectory $\phi(t)$ for $t \in P_{\text{time}}$, the goal is to estimate the parameter μ and initial condition $\mathbf{x}_0$ that best reproduce the observed trajectory under the DeepONet prediction $\hat{\phi}(\mu, \mathbf{x}_0)(t)$.

Let $P_{\text{time}} = \{t_1, \dots, t_{N_t}\}$ and $\phi(P_{\text{time}}) = \{\phi_1, \dots, \phi_{N_t}\}$, with $\phi_i = \phi(t_i)$. Then, the inverse problem consists in solving the optimization problem

$$(\mu^*, \mathbf{x}_0^*) = \arg \min_{\mu, \mathbf{x}_0} L_{\text{IP}}(\mu, \mathbf{x}_0) = \arg \min_{\mu, \mathbf{x}_0} \sum_{i=1}^{N_t} \left\| \hat{\phi}(\mu, \mathbf{x}_0)(t_i) - \phi_i \right\|^2, \quad (40)$$

with a possible scaling inside the norm to balance the contribution of the different variables.

Thus, using reverse-mode automatic differentiation in the DeepONet, we can compute the gradient of the inverse problem loss L_{IP} and apply gradient-based optimization (Adam) to obtain estimates of the parameters μ and the initial condition $\mathbf{x}_0$, in analogy with the optimization performed during network training. Additionally, by considering multiple initializations, we can perform a parallel search by running gradient descent for N_{paths} independent trajectories, each starting from a different initial guess, and then select the solution that yields the lowest value of L_{IP} .

This is a major advantage of DeepONet-based inference, since the inverse problem can be formulated directly on observed trajectories that may deviate from the true dynamics due to noise or modelling errors.

5.6 DeepONet Experimentation

In this section, we apply the previously described DeepONet methodology to approximate the solution of the phase-amplitude coupling IVP (24). We describe the specific adaptations of the DeepONet framework to this specific IVP, together with the selected hyperparameters, architectural features, and explored dynamical regimes considered during training. We conclude the section with an analysis of the obtained approximations, their comparison with numerically integrated reference trajectories, and the performance of the inverse problem prediction.

All the code developed for the implementation, training and prediction of the DeepONet model is available in the associated GitHub repository.

5.6.1 Considered Phase-Amplitude Coupling Dynamics Initial Value Problem

We now introduce the specific initial value problem associated with the phase-amplitude dynamics (24) considered in this experiment.

We study a perturbation of the homogeneous periodic solution corresponding to $I_{\text{ext}}^E = 10$ and $\varepsilon = 0$. We note that such a solution exists, as can be observed in Figure 7, where the pair $(I_{\text{ext}}^E, \varepsilon) = (10, 0)$ corresponds

to a stable periodic orbit. Its period T and largest non-zero Floquet exponent λ can be computed using the algorithm described in Section 4.3, and are reported in Table 2.

The considered IVP is associated with the phase–amplitude dynamics described by system (24) for the given values of T and λ . The functions (25) describing the coupling effects are approximated via Fourier expansions and are available in the accompanying GitHub repository.

5.6.2 DeepONet Architecture Refinements for Phase-Amplitude Coupling Dynamics

To approximate the IVP solution associated with the phase-amplitude coupling system (24), we introduce several additional refinements to the DeepONet architecture.

We observe that the solution of this system for $\varepsilon = 0$ is analytic:

$$\phi(\varepsilon = 0, \theta_0, \sigma_0)(t) \equiv \phi_b(\theta_0, \sigma_0)(t) = (\theta_b(t), \sigma_b(t)), \quad (41)$$

where

$$\theta_b(t) = t + \theta_0, \quad \sigma_b(t) = \sigma_0 \exp(\lambda t). \quad (42)$$

We refer to this solution as the *baseline solution*. Instead of training the DeepONet to directly learn the full solution for $\varepsilon > 0$, we train it to learn the deviation from the baseline solution (42). This improves the approximation performance and facilitates the training process.

To implement this idea, we modify the output of the DeepONet. Rather than using the raw trunk-branch approximation (29), we define the predicted solution as

$$\hat{\phi}(\varepsilon, \theta_0, \sigma_0)(t) = \phi_b(\theta_0, \sigma_0)(t) + \varepsilon(1 - \exp(-t)) \sum_{i=1}^P T_i(t) \mathbf{B}_i(\varepsilon, \theta_0, \sigma_0). \quad (43)$$

We note that this construction automatically recovers the baseline solution when $\varepsilon = 0$, while for $\varepsilon > 0$ the DeepONet learns the correction relative to the baseline solution.

In the refined DeepONet output (43), we also include the time-dependent factor $1 - \exp(-t)$ so that the initial condition is automatically satisfied at $t = 0$. Consequently, the initial condition loss does not need to be included explicitly, and the DeepONet is trained by minimizing the total loss

$$L_{\text{total}} = L_{\text{superv}} + \gamma L_{\text{ODE}}, \quad (44)$$

where

$$L_{\text{superv}} = \left\| \hat{\theta}(\varepsilon, \theta_0)(t) - \theta(\varepsilon, \theta_0)(t) \right\|^2 + \beta \left\| \hat{\sigma}(\varepsilon, \sigma_0)(t) - \sigma(\varepsilon, \sigma_0)(t) \right\|^2, \quad (45)$$

and $\gamma, \beta \geq 0$ are relative weights associated with each loss contribution. We note that setting $\gamma = 0$ corresponds to using only the supervised loss, while $\beta > 0$ controls the relative importance assigned to the σ variable during training.

5.6.3 Hyperparameter Selection and Software Implementation

In this part, we present the specifications of the DeepONet software implementation used in the experiment.

Let us describe Table 2. First, we remark that the DeepONet architecture hyperparameters were tuned manually, without attempting to determine an "absolute optimal" configuration. Secondly, the DeepONet has been trained to predict the IVP solution over the specified ranges of t , ε , θ_0 , and σ_0 . These intervals could be modified depending on the experimental setting.

The network is trained using supervised data consisting of numerically integrated trajectories generated within these variable ranges. The training dataset is constructed from randomly sampled parameter and initial condition pairs, together with corresponding sampled time points within the specified ranges. We note that we used a relatively small number of supervised trajectories evaluated at a limited number of time points.

Observe that no physics-informed contribution was included, since $\gamma = 0$. It could be of interest to include a physics-informed loss and compare the results with those obtained using purely supervised training; however, this lies outside the scope of this project. Additionally, note that a larger weight β was assigned to the supervised loss associated with the σ variable, since its variability with respect to the parameter ε and the initial conditions is significantly larger than that of θ .

We implemented the DeepONet architecture, training procedure, and prediction pipeline in Python using the JAX library, which provides efficient deep learning tools for constructing the considered DeepONet architecture and is highly optimized for large-scale numerical computations and network training. The supervised data were numerically integrated using the Diffrax library.

DeepONet architecture	
Basis size p	64
Hidden dimension	64
Latent dimension	128
Depth	4
Fourier features F	0.1, 0.25, 0.5, 1.0, 2.0, 3.0
Explored dynamics	
T	21.36555175500626
λ	-0.15358058186323872
t range	$[0, T]$
ε range	$[0, 1]$
θ_0 range	$[0, \frac{T}{4}]$
σ_0 range	$[-1, 1]$
Training	
Number of trajectories $ D_{traj} $	1600
Number of timesteps $ D_{time} $	128
γ	0
β	10
Learning rate η	10^{-4}
Batch size S_{batch}	128
Number of epochs N_{epoch}	25 000

Table 2: DeepONet architecture, explored dynamics, supervised dataset, and training hyperparameters.

5.6.4 Experiment Results

In this part, we present the results obtained for the considered experimental setting, including the training loss convergence and representative examples of the DeepONet prediction performance.

Figure 13 displays the loss evolution during the training process. We observe a sudden decrease in the training loss at the beginning. During the first 200 000 steps, the noise associated with the stochastic optimization process appears to scale proportionally with the loss value, since its amplitude remains relatively uniform on the logarithmic scale throughout training. Additionally, the continuous decrease in the loss during this stage indicates that the DeepONet continues learning throughout the training process.

At later stages (beyond 200 000 steps), the convergence slows down and the stochastic noise increases relative to the loss value, as reflected by its larger amplitude on the logarithmic scale.

Figures 14, 15, and 16 show comparisons between the trajectories predicted by the DeepONet and the numerically integrated trajectories for different random choices of ε , θ_0 , and σ_0 . The first plots display the predicted phase trajectory $\hat{\theta}(t)$, while the second ones show the predicted amplitude trajectory $\hat{\sigma}(t)$.

By exploring different parameter and initial condition selections, one can observe that some trajectories are approximated more accurately than others, without any clear a priori pattern. The approximation accuracy could potentially be further improved by modifying the hyperparameter configuration, changing the considered loss functions, or increasing the number of supervised trajectories and training epochs.

Also, observe that ε and the initial conditions were chosen within the ranges considered during training. Predictions generally deteriorate for values outside these intervals.

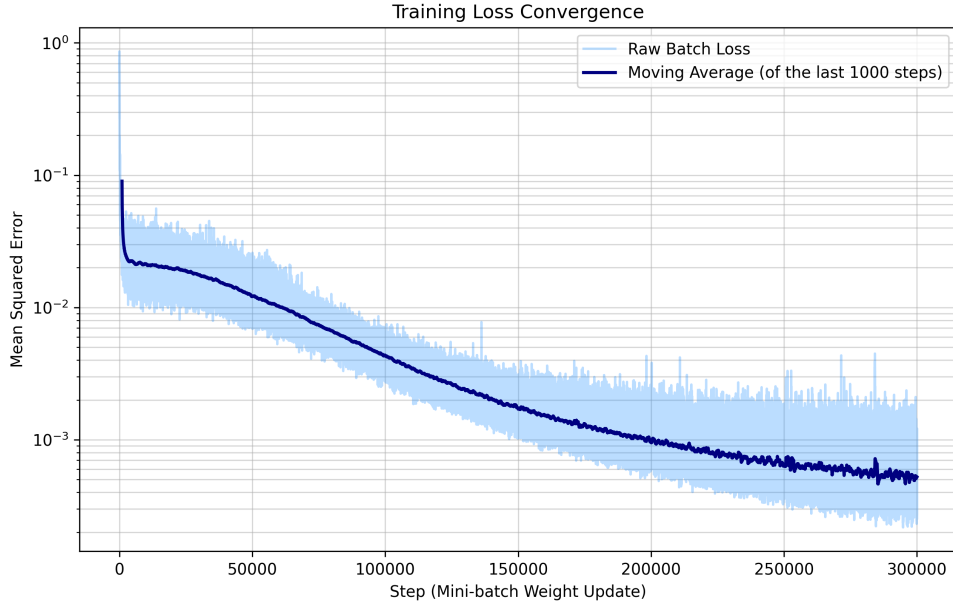


Figure 13: Loss convergence during the training process on a logarithmic scale. The training loss is plotted for each mini-batch update of the network parameters, together with a moving average computed over the previous 1000 steps at each iteration.

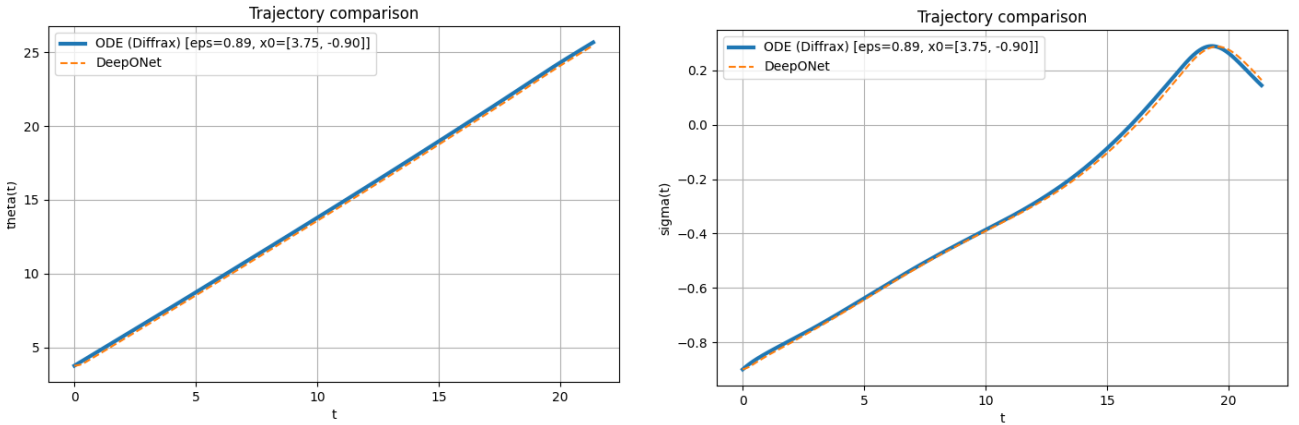


Figure 14: Comparison between the numerically integrated trajectories (blue) and the DeepONet-predicted trajectories (orange), corresponding to the solution of the phase-amplitude system (24) for $\varepsilon = 0.89$.

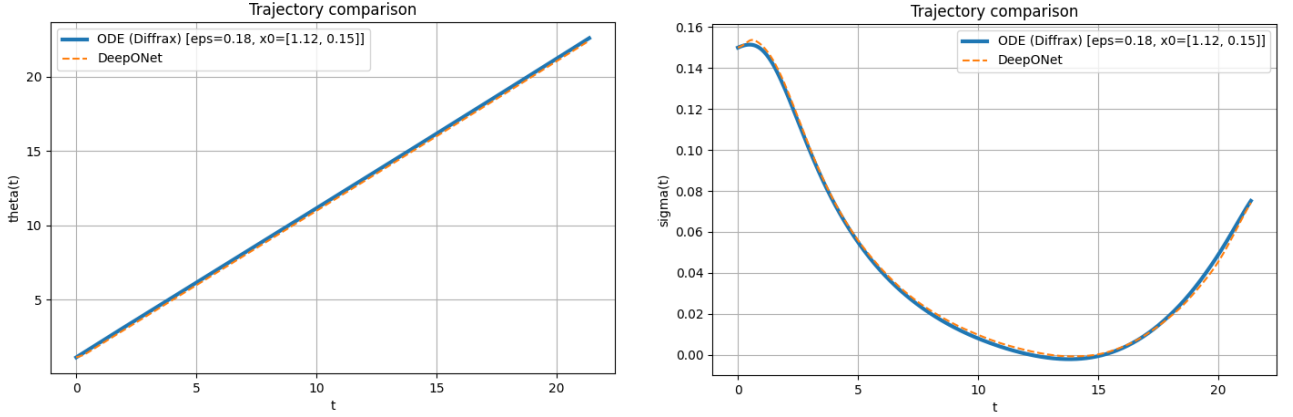


Figure 15: Comparison between the numerically integrated trajectories (blue) and the DeepONet-predicted trajectories (orange), corresponding to the solution of the phase-amplitude system (24) for $\varepsilon = 0.18$.

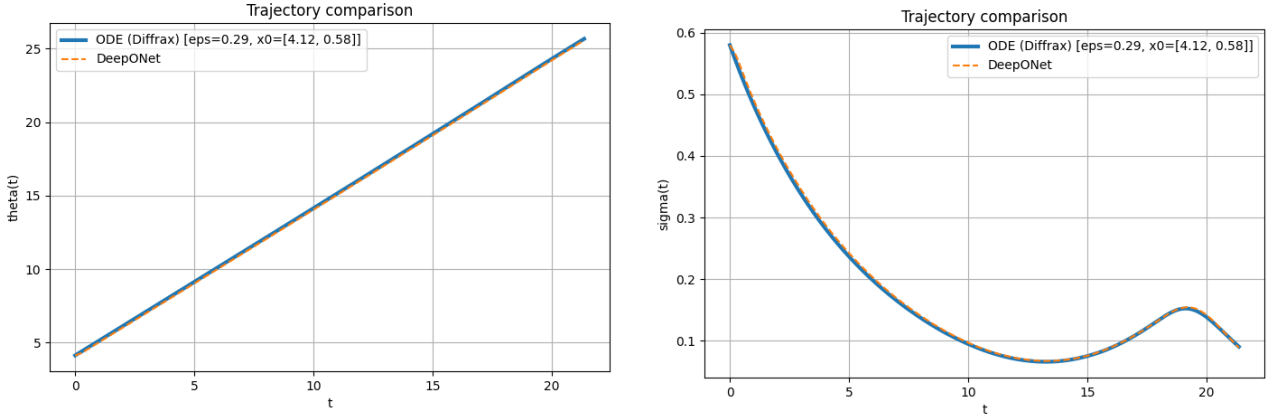


Figure 16: Comparison between the numerically integrated trajectories (blue) and the DeepONet-predicted trajectories (orange), corresponding to the solution of the phase-amplitude system (24) for $\varepsilon = 0.29$.

Figures 17, 18, and 19 show the DeepONet performance in the inverse problem setting, considering as input the trajectories from Figures 14, 15, and 16, respectively, with added measurement noise. We observe that, even when the available data points from the input trajectory $\sigma(t)$ are noticeably noisy, the inverse problem predictions remain mostly accurate, although the accuracy generally decreases as the noise level increases, for instance in the ε prediction. The predicted values and the associated relative errors are included in Table 3.

We remark that this is an important predictive capability of the DeepONet model, which is useful for fitting noisy trajectory data to the dynamical model considered during training.

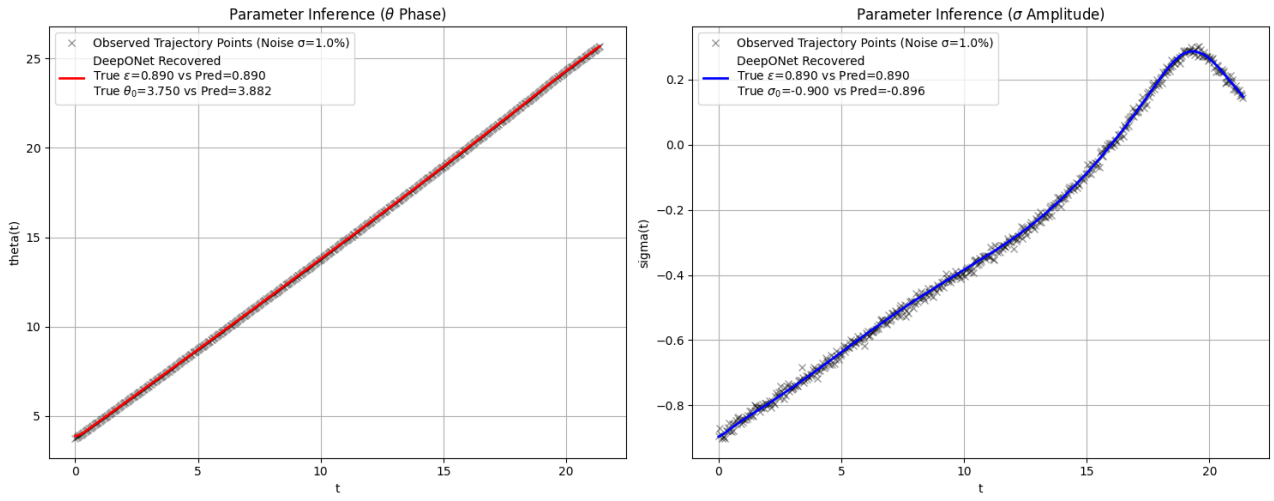


Figure 17: DeepONet-predicted trajectories obtained from the inverse problem, using as input the numerically integrated trajectories considered in Figure 14 with added measurement noise.

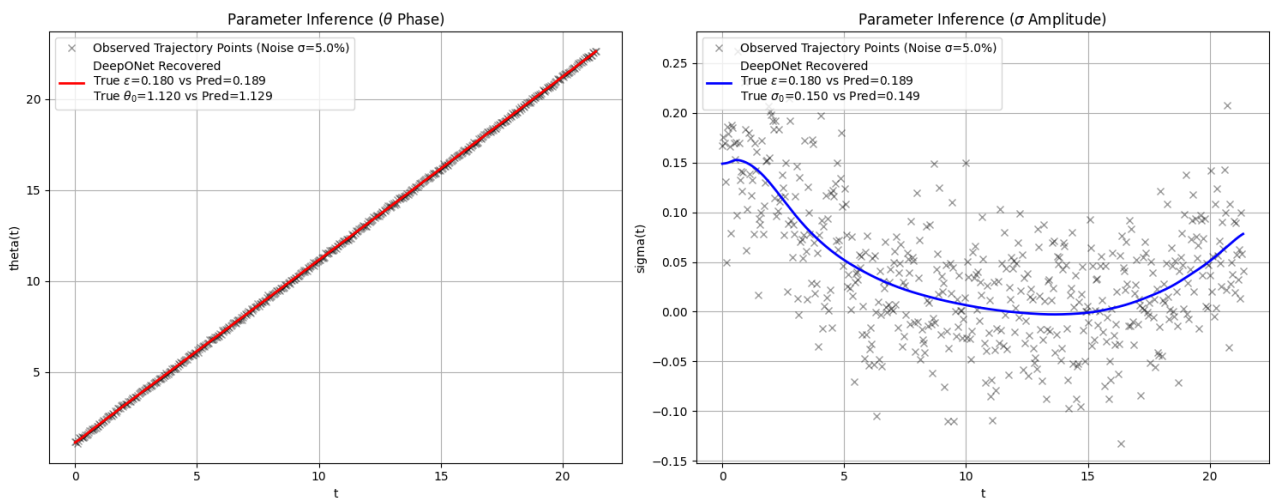


Figure 18: DeepONet-predicted trajectories obtained from the inverse problem, using as input the numerically integrated trajectories considered in Figure 15 with added measurement noise.

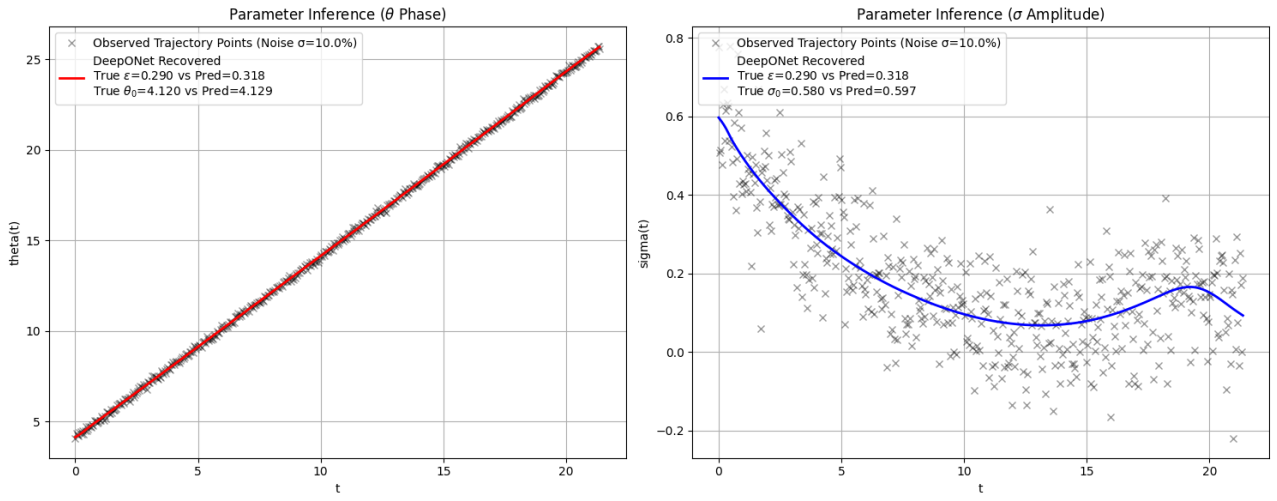


Figure 19: DeepONet-predicted trajectories obtained from the inverse problem, using as input the numerically integrated trajectories considered in Figure 16 with added measurement noise.

	Parameter	True Value	Predicted Value	Absolute Error	Relative Error (%)
Case 1 (Noise = 1.0%)	ϵ	0.890	0.890	0.000	0.00%
	θ_0	3.750	3.882	0.132	3.52%
	σ_0	-0.900	-0.896	0.004	0.44%
Case 2 (Noise = 5.0%)	ϵ	0.180	0.189	0.009	5.00%
	θ_0	1.120	1.129	0.009	0.80%
	σ_0	0.150	0.149	0.001	0.67%
Case 3 (Noise = 10.0%)	ϵ	0.290	0.318	0.028	9.66%
	θ_0	4.120	4.129	0.009	0.22%
	σ_0	0.580	0.597	0.017	2.93%

Table 3: Parameter inference values obtained from the DeepONet inverse problem predictions, together with absolute and relative errors across the three considered cases.

Appendices

A. K-Means Clustering Method

In this appendix, we introduce the k-means clustering method.

Consider a finite set D , typically a dataset, contained in some space X . By identifying each element with a label, we may represent D as the set $\{1, \dots, N\}$, where $N = |D|$ denotes its cardinality.

A k -clustering of D is defined as a partition into k subsets, $\mathcal{C} = \{C_1, \dots, C_k\}$, which are pairwise disjoint and whose union is the whole set D . Clustering groups points that share some notion of similarity, for instance, being close to each other in a given representation space (here, the space X equipped with a distance d).

The k -means algorithm is a non-hierarchical clustering method that minimizes the total sum of squared distances (SSD) for a k -clustering $\mathcal{C}$, that is,

$$\mathcal{C}^* = \arg \min_{\mathcal{C}} SSD(\mathcal{C}), \quad (46)$$

$$SSD(\mathcal{C}) = \sum_{i=1}^k SSD(C_i) = \sum_{i=1}^k \sum_{x, y \in C_i} d(x, y)^2. \quad (47)$$

for $x, y \in X$, where (X, d) is a metric space. This formulation is natural, as it seeks to minimize the pairwise dispersion within each cluster.

When the metric space is Euclidean, $(X, d) = (\mathbb{R}^n, \|\cdot\|)$, it can be shown that

$$SSD(\mathcal{C}) = \sum_{x, y \in \mathcal{C}} d(x, y)^2 = 2|\mathcal{C}| \sum_{x \in \mathcal{C}} d(x, m_{\mathcal{C}})^2, \quad (48)$$

where $m_{\mathcal{C}} = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} x$ is the arithmetic mean of the cluster, which we refer to as its centre. Then, the optimization problem (46) can be reframed as

$$\mathcal{C}^* = \arg \min_{\mathcal{C}} \sum_{i=1}^k \sum_{x \in C_i} d(x, m_{C_i})^2. \quad (49)$$

An algorithm to optimize (49) can be described as follows:

Lloyd's algorithm

1. Initialize a random k -clustering $\mathcal{C} = \{C_1, \dots, C_k\}$ of the set D .
2. Compute the arithmetic mean of each cluster, m_{C_i} .

3. Assign each element $x \in D$ to the cluster with nearest centre: if

$$i(x) = \arg \min_{1 \leq i \leq k} d(x, m_{C_i}),$$

then assign x to the cluster $C_{i(x)}$.

4. Repeat steps 2 and 3 until the assignments no longer change.

Observe that the output of the algorithm depends on the initial clustering in Step 1, since the optimization problem (49) is, in general, non-convex. Some initialization strategies, such as k -means++, select well-spaced initial centres, reducing the likelihood of poor clustering results and improving convergence speed and stability.

Additionally, we can use the *Adjusted Rand Index* (ARI) as a metric to measure the stability of clustering across multiple runs with different random initializations.

Specifically, given two independently initialized clusterings, the ARI compares pairs of data points and checks whether they are assigned consistently in both clusterings ($\text{ARI} = 1$), whether the agreement is no better than random ($\text{ARI} = 0$), or worse than random ($\text{ARI} < 0$). By averaging over different pairs and multiple initializations, we obtain a stability measure for the considered clustering method.

B. Clustering Metrics

In this appendix, we briefly present the clustering quality metrics used in the spectral clustering considered in the project.

B.1 Eigengap Heuristic

One tool which is particularly designed for spectral clustering is the eigengap heuristic, which aims to select the number k of clusters such that the eigenvalues $\lambda_1, \dots, \lambda_k$ of the Laplacian are small, but λ_{k+1} is relatively large (thus creating a gap between the eigenvalues).

An intuitive justification is based on perturbation theory, where we observe that in the ideal case of k completely disconnected clusters, the eigenvalue 0 has multiplicity k , and there is then a gap to the $(k + 1)$ -th eigenvalue $\lambda_{k+1} > 0$.

The eigengap heuristic usually works well if the data contains well-pronounced clusters, but in ambiguous cases it may also return ambiguous results. For instance, for dense structural connectivities, there is no clear leading eigengap.

B.2 Q-Modularity

For the quality assessment of graph clustering, a relevant metric is Q -modularity. This quality function measures how good a clustering is in terms of the number of edges located within each cluster in contrast to those between clusters.

More precisely, Q -modularity compares, for each cluster C_k , the actual connections of a graph, W_{ij} , with the expected connections under a random graph model, $\frac{d_i d_j}{2m}$, where d_i denotes the degree of node i and m the total number of edges in the graph.

The Q -modularity for the whole clustering can be computed as

$$Q = \frac{1}{2|E|} \sum_{1 \leq i, j \leq N} \left(W_{ij} - \frac{d_i d_j}{2|E|} \right) \mathbf{1}_{\{C_i = C_j\}}. \quad (50)$$

Values of Q close to zero indicate the absence of cluster structure (i.e., random-like connections). A value $Q > 0$ suggests the presence of meaningful structure, while $Q \sim 0.3 - 0.7$ typically indicates strong clustering structure.

B.3 Silhouette Width

The silhouette width $s(i)$ is a general clustering metric based on the dissimilarities between an element $i \in \{1, \dots, N\}$ and its own cluster and neighbouring cluster.

We define the silhouette width $s(i)$ as follows:

- Let $a(i)$ be the average dissimilarity between i and all other points in the cluster to which i belongs (if i is the only observation in its cluster, $s(i)$ is defined as 0).
- For each other cluster C_k , let $d(i, C_k)$ be the average dissimilarity between i and all observations in C_k .
- Define $b(i) = \min_k d(i, C_k)$, which can be interpreted as the dissimilarity between i and its “neighbouring” cluster, i.e., the closest cluster to which it does not belong.

- Finally,

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}. \quad (51)$$

Observations with a large $s(i)$ (close to 1) are well clustered, values near 0 indicate that the observation lies between two clusters, and negative values suggest that the observation may have been assigned to the wrong cluster.

B.4 Calinski-Harabasz Index

The Calinski–Harabasz index CH measures the quality of a clustering by comparing the separation between clusters with the compactness within clusters.

We define the Calinski–Harabasz index CH as follows:

- The average within-cluster sum of squared distances is defined as

$$W_{\text{avg}} = \sum_{l=1}^k \frac{1}{|C_l|} \sum_{i,j \in C_l} d_{ij}^2.$$

- The average between-cluster sum of squared distances is

$$B_{\text{avg}} = \frac{1}{N} \sum_{i,j=1}^N d_{ij}^2 - W_{\text{avg}}.$$

- The Calinski–Harabasz index is given by

$$\text{CH} = \frac{B_{\text{avg}}/(k-1)}{W_{\text{avg}}/(n-k)}. \quad (52)$$

A larger value of CH indicates that clusters are well separated (large between-cluster dispersion) and internally cohesive (small within-cluster dispersion), which is desirable. Conversely, a small value of CH suggests that clusters are either not well separated or are too dispersed internally.

C. Training Techniques

In this appendix, we introduce the techniques used in this project during the training process.

C.1 Adam Optimization Algorithm

A widely used variant of gradient descent is the Adaptive Moment Estimation (Adam), an optimization method that computes adaptive learning rates at each iteration. Specifically, it stores exponentially decaying averages of past gradients and squared gradients. At each iteration, these quantities are updated as follows:

$$m_{k+1} = \beta_1 m_k + (1 - \beta_1) \nabla_{\Theta} L(\Theta_k), \quad (53)$$

$$v_{k+1} = \beta_2 v_k + (1 - \beta_2) (\nabla_{\Theta} L(\Theta_k))^2, \quad (54)$$

where m_k and v_k estimate the first moment (mean) and the second moment (uncentered variance), respectively, and β_1 and β_2 are the decay rates. The square $(\nabla_{\Theta} L(\Theta_k))^2$ is understood element-wise.

Since m_k and v_k are initialized as zero, they are biased towards zero, especially during the initial iterations. For this reason, we use their bias-corrected versions:

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}. \quad (55)$$

The parameter update is then given by

$$\Theta_{k+1} = \Theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon}, \quad (56)$$

where $\varepsilon > 0$ is a small constant used to avoid division by zero.

By accumulating estimates of the first and second moments of the gradients, Adam adapts the effective step size in each coordinate, accelerating convergence in directions of consistent descent while damping oscillations in directions of high curvature. This makes the optimization more stable in ill-conditioned regions of the loss landscape, such as ravines, where the curvature differs significantly across dimensions.

Additionally, the momentum term helps the iterates maintain consistent descent directions, which can improve traversal of flat regions and reduce the likelihood of becoming trapped in saddle points or shallow local minima.

We note that the Adam algorithm introduces a few free parameters. The decay rates are typically fixed to the default values $\beta_1 = 0.9$ and $\beta_2 = 0.999$, whereas $\varepsilon = 10^{-8}$. The learning rate η is a hyperparameter that depends on the considered problem.

C.2 Gradient Clipping

An additional modification of the optimization algorithm is gradient clipping at each iteration. That is, instead of using the raw gradient $g = \nabla_{\Theta} L(\Theta)$, we rescale it as

$$\tilde{g} = \min\left(1, \frac{C}{\|g\|}\right) g. \quad (57)$$

During training, this helps prevent excessively large parameter updates in steep regions of the loss landscape while preserving the direction of the gradient. Typically, we take $C = 1$ as the clipping threshold.

C.3 Xavier/Glorot Initialization

We have commented how the effective learning step is adapted through the Adam algorithm. Another important factor to consider is parameter initialization. A standard choice is the Xavier/Glorot initialization, which initializes independently the parameters of each dense neural network layer according to

$$\omega_{ij} \sim \mathcal{U} \left(-\sqrt{\frac{6}{n+m}}, \sqrt{\frac{6}{n+m}} \right), \quad (58)$$

$$b_i = 0, \quad (59)$$

where $\sim \mathcal{U}$ denotes sampling from a uniform distribution, and n and m are the dimensions of the input and output layers, respectively.

The Xavier/Glorot initialization is designed to approximately preserve the variance of activations across layers when activations are symmetric and centred around zero, which is the case for the tanh activation function used in this work. This helps prevent activations from exploding or vanishing too quickly at initialization.

References

- [1] Delicado-Moll, Rosa Maria, Gemma Huguet, and Pau Clusella. 'Emergent Spatiotemporal Dynamics in Large-Scale Brain Networks with next Generation Neural Mass Models'. *Physica D: Nonlinear Phenomena* 493 (September 2026): 135232.
- [2] S. Coombes and A. Byrne. *Next generation neural mass models*. Springer, pages 1–16, 2019.
- [3] E. Montbrió, D. Pazó, and A. Roxin. Macroscopic description for networks of spiking neurons. *Physical Review X*, 5(2):021028, 2015.
- [4] Luxburg, Ulrike von. 'A Tutorial on Spectral Clustering'. arXiv:0711.0189. Preprint, arXiv, 1 November 2007.
- [5] Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer, 2009.
- [6] Newman, M. E. J. 'Modularity and Community Structure in Networks'. *Proceedings of the National Academy of Sciences* 103, no. 23 (2006): 8577–82.
- [7] Deco, Gustavo, Joana Cabral, Mark W. Woolrich, Angus B. A. Stevner, Tim J. Van Hartevelt, and Morten L. Kringelbach. 'Single or Multiple Frequency Generators in On-Going Brain Activity: A Mechanistic Whole-Brain Model of Empirical MEG Data'. *NeuroImage* 152 (May 2017): 538–50.
- [8] Van Mieghem, Piet. *Graph Spectra for Complex Networks*. Cambridge University Press, 2011.
- [9] David Reyner-Parra, Alberto Pérez-Cervera, and Gemma Huguet. Efficient numerical algorithms for phase-amplitude reduction on the slow attracting manifold of limit cycles. *Journal of Nonlinear Science*, 36(2), 2026.
- [10] Lu, Lu, Pengzhan Jin, and George Em Karniadakis. 'DeepONet: Learning Nonlinear Operators for Identifying Differential Equations Based on the Universal Approximation Theorem of Operators'. *Nature Machine Intelligence* 3, no. 3 (2021): 218–29.
- [11] T. Chen and H. Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE Transactions on Neural Networks*, 6(4):911–917, 1995.
- [12] Baydin, Atilim Gunes, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. 'Automatic Differentiation in Machine Learning: A Survey'. arXiv:1502.05767. Preprint, arXiv, 5 February 2018.
- [13] Ruder, Sebastian. 'An Overview of Gradient Descent Optimization Algorithms'. arXiv:1609.04747. Preprint, arXiv, 15 June 2017.
- [14] Wang, Sifan, Hanwen Wang, and Paris Perdikaris. 'Learning the Solution Operator of Parametric Partial Differential Equations with Physics-Informed DeepONets'. *SCIENCE ADVANCES*, 2021.